Design, Cross-Verification, and Statistically Qualified Performance Evaluation of a PDSCH-Oriented 5G NR Massive MIMO-OFDM Physical-Layer Link-Level Simulator

Md Moklesur Rahman

Independent Researcher, Finland | moklesur.eee@gmail.com | github.com/dipucwc

Abstract

Link-level simulation is the standard evaluation method of physical-layer research, yet published simulation studies rarely document how the correctness of the underlying simulator was established or how statistically reliable the reported Monte Carlo estimates are. This paper presents a PDSCH-oriented 5G NR Massive MIMO-OFDM link-level simulator together with the two methodological elements that qualify its results for quantitative use: a gate-based verification methodology that anchors the implementation to closed-form references before any performance figure is accepted, and a tolerance-based cross-verification protocol under which an independently written, uncoded, CRC-protected Python reference implementation reproduces the MATLAB campaign metric by metric under explicit statistical tolerances. The MATLAB platform implements the complete coded and uncoded study, from transport-block generation through TS 38.212 LDPC coding, wideband eigenbeamforming across a 64×8 antenna configuration with four spatial layers, DM-RS-based least-squares estimation of the effective channel, and unbiased MMSE equalization, to CRC-verified block decisions, and the verified functions are additionally executed as a Simulink testbench, a third execution environment. All thirty-five MATLAB-Python metric comparisons pass their tolerances, and the seven-point Simulink sweep reproduces the reference campaign at every grid point. The principal uncoded campaign uses adaptive Monte Carlo stopping, running each SNR point until 500 bit errors or 100 block errors accumulate within a budget of 500 to 20,000 frames, and every reported estimate carries an exact Clopper-Pearson 95% confidence interval computed from the recorded error counts, with zero-error operating points reported as upper confidence bounds rather than as zeros, down to a bit error rate below 6.7×10^{-9} from 5.5×10^{8} evaluated bits per point. The protocol demonstrably detects real defects: an underspecified interpolation edge condition, statistically invisible at coarse tolerances, was localized to six subcarriers by a paired common-input experiment, corrected, and re-verified, with the largest cross-implementation deviation collapsing from 0.27 to 0.046 decades. The executed campaigns, which use a 12-resource-block reduced allocation to bound the cost of the 64×8 study, measure a combined array-dimension and eigenbeamforming gain of 19.6 dB at a bit error rate of 10^{-1} over an unprecoded 4×4 reference at equal total transmit power, and rate-ordered LDPC waterfalls whose throughput plateaus match the transport-block arithmetic exactly.

Keywords: 5G NR, PDSCH, Massive MIMO, link-level simulation, verification methodology, Monte Carlo confidence intervals.

1. Introduction

Mobile networks carry more traffic every year, and the 3GPP 5G New Radio (NR) physical layer addresses this growth with scalable numerology, LDPC channel coding, high-order modulation, and Massive Multiple-Input Multiple-Output (MIMO) transmission built on Orthogonal Frequency Division Multiplexing (OFDM) [1]-[4]. The Physical Downlink Shared Channel (PDSCH) is the main downlink data channel, and its performance emerges from a long chain of tightly coupled processing stages: transport-block processing, channel coding, modulation, precoding, resource mapping, OFDM waveform generation, the wireless channel, channel estimation, equalization, soft demodulation, and channel decoding. None of these stages can be evaluated in isolation, and link-level simulation is therefore the standard evaluation method of physical-layer engineering: it exercises the complete chain under controlled, repeatable, and reproducible conditions before any hardware exists.

Two practical weaknesses recur in link-level studies. The first concerns correctness. A simulator is itself a piece of engineering, and a subtle error in noise calibration, transform scaling, or symbol mapping shifts every curve it produces; results are frequently published without documenting how the implementation was verified. The second concerns statistical rigor. Monte Carlo estimates of the bit error rate (BER) and the block error rate (BLER) are binomial proportions computed from finite trial counts, yet they are commonly reported as bare point values, and an observed error count of zero is reported as an error rate of zero, a statement the executed sample size cannot support. Both weaknesses undermine the quantitative claims, such as coding gain, SNR gain at a target BLER, or one algorithm outperforming another, that link-level studies are intended to support.

This paper addresses both weaknesses in the context of a complete, executable simulator. The system under study is a PDSCH-oriented 5G NR Massive MIMO-OFDM link-level simulator whose MATLAB platform covers CRC attachment, TS 38.212 LDPC coding and rate matching, Gold-sequence scrambling, QPSK to 256-QAM modulation, layer mapping, wideband eigenbeamforming precoding, DM-RS insertion, CP-OFDM generation, DM-RS-based least-squares (LS) estimation of the effective layer channel, zero-forcing (ZF) and unbiased minimum mean-square error (MMSE) equalization, max-log soft demodulation, LDPC decoding, and CRC verification. The evaluated configurations reach a 64×8 antenna array with four spatial layers over AWGN, Rayleigh, Rician, and 3GPP TDL channels. Every algorithm of the paper is presented as a formal derivation, as a numbered procedure with a workflow diagram, and as a traceability mapping to the MATLAB function, the Python module, and the Simulink block that implement it, so that the mathematics, the code of the three execution environments, and the reported results remain aligned throughout.

1.1 Contributions and paper organization

The contributions are fourfold. First, the paper formalizes a gate-based verification methodology in which quantitative acceptance gates, exact constellation power,

machine-precision OFDM round-trip error, CRC accept and reject behavior, closed-form BER references in AWGN and Rayleigh fading, and an exact MIMO capacity identity, must pass before any performance campaign is allowed to run, and the acceptance of every result is expressed as a function of five stored artifacts so that each figure is regenerable from its provenance. Second, the paper defines a tolerance-based cross-verification protocol between two independently written implementations of the same mathematical specification: the complete MATLAB platform and an uncoded, CRC-protected Python reference implementation that independently realizes the MIMO-OFDM processing, LS estimation, ZF and unbiased-MMSE detection, and metric chain. The comparison is statistical and metric-level, using a logarithmic decade metric for error-rate quantities and relative or absolute tolerances for smooth quantities; the executed record contains thirty-five comparisons, all passing. The verified functions are additionally executed inside a Simulink testbench, a third execution environment whose seven-point SNR sweep reproduces the reference campaign at every grid point. Third, the paper executes adaptive Monte Carlo stopping on the principal uncoded campaign and attaches exact Clopper-Pearson confidence intervals, computed from the recorded bit and block error counts, to every reported error-rate estimate, reporting zero-error operating points as upper confidence bounds; this converts the customary bare Monte Carlo point estimates into statistically defensible statements. Fourth, the executed evaluation itself quantifies the massive-array benefit, a measured 19.6 dB combined array-dimension and eigenbeamforming gain at BER 10^{-1} for the 64×8 configuration against an unprecoded 4×4 reference at equal total transmit power, the measured cost of real channel estimation relative to ideal channel state information (CSI), and rate-ordered LDPC BLER waterfalls whose throughput plateaus match the transport-block arithmetic exactly.

The remainder of the paper is organized as follows. Section 2 positions the work against related simulators and methods. Section 3 states the system model, and Section 4 derives the receiver algorithms and presents them as numbered procedures with their workflow diagram and their implementation traceability. Section 5 describes the simulator architecture and its three execution environments. Section 6 presents the verification methodology, and Section 7 the statistical qualification of the Monte Carlo estimates. Section 8 reports and discusses the executed results, Section 9 provides the complexity analysis and the reproducibility package, Section 10 states the limitations and future work, and Section 11 concludes.

2. Related Work

This section reviews the existing link-level simulation frameworks and the underlying algorithmic literature, and positions the contribution of this paper against both.

2.1 Link-level simulation frameworks

Several substantial link-level simulation frameworks precede this work. The MATLAB 5G Toolbox [12] provides standard-compliant NR waveform, transport-channel, and channel-model components together with reference PDSCH examples, and this platform deliberately reuses its DL-SCH encoder, decoder, and symbol mapping in the coded campaigns so that the TS 38.212 processing is anchored to a maintained standard-compliant implementation. The Vienna 5G Link Level Simulator [14] is an academic MATLAB framework covering flexible numerology, coding, and multi-antenna processing for standard-oriented research. Sionna [15] is an open-source, GPU-accelerated Python library built on TensorFlow that supplies differentiable physical-layer components for machine-learning-oriented research, and OpenAirInterface [16] provides a real-time software implementation of the 3GPP stack. These frameworks are broader than the present platform in feature coverage; none of them, to the best of the author's knowledge, publishes a dual-implementation cross-verification record in which two independently written codebases of the same specification are compared metric by metric under stated statistical tolerances, nor do their published examples attach exact confidence intervals to every reported error-rate point. The methodological contribution of this paper lies in these two elements rather than in feature coverage.

2.2 Algorithmic foundations

The algorithmic components rest on established literature. Massive MIMO and its array gains were established by Marzetta [17] and surveyed by Larsson et al. [18], Rusek et al. [19], and Björnson et al. [20]; the capacity reference of this paper is Telatar's classical result [7]. Pilot-based OFDM channel estimation by LS and by model-aware linear MMSE derives from van de Beek et al. [21] and Edfors et al. [22], with pilot-arrangement trade-offs surveyed by Coleri et al. [23]; the estimator executed here is the LS family applied to the precoded effective channel that NR DM-RS makes observable [2]. The 5G NR LDPC design is described by Richardson and Kudekar [24], and limited-feedback and codebook precoding, the practical relative of the wideband eigenbeamformer used here, is surveyed by Love et al. [25]. On the methodological side, Jeruchim's treatment of Monte Carlo BER estimation [29] and the exact binomial interval of Clopper and Pearson [27] underpin Section 7, and the case for reproducible computational research in signal processing was made by Vandewalle et al. [26].

2.3 Positioning of the present work

Table 1 summarizes the positioning. The first column lists the compared aspects, the following three columns state the published status of each aspect for the MATLAB 5G Toolbox, the Vienna 5G Link Level Simulator, and Sionna, and the final column states the

corresponding property of this work. The feature rows for the external frameworks reflect their published documentation, and the comparison claims are limited to what those publications state.

Table 1. Positioning against representative link-level frameworks, based on published documentation (n/p: not published).

Aspect	**5G Toolbox [12]**	**Vienna 5G LLS [14]**	**Sionna [15]**	**This paper**
Language / environment	MATLAB	MATLAB	Python (TF)	MATLAB, Python, Simulink
Standard-compliant coded chain	Yes	Yes	Yes	Yes (5G Toolbox DL-SCH)
Dual-implementation cross-verification with published tolerance record	n/p	n/p	n/p	Yes (35/35)
Verification gates executed before every campaign	n/p	n/p	n/p	Yes
Exact confidence intervals on every reported error rate	n/p	n/p	n/p	Yes
Executed 64×8 eigenbeamformed profile with stored provenance	Example-specific	Config.-specific	Config.-specific	Yes
Simulink testbench of the same verified functions	No	No	No	Yes

The scope of the comparison is intentionally limited. The external frameworks are mature, feature-rich, and in several respects more capable than the present platform. The claim of this paper is methodological: verification evidence and statistical qualification should accompany published link-level results, and the platform applies this practice end to end on a large 64-antenna configuration.

3. System Model

The mathematical model is stated in the order in which the implementations execute it, and each equation is accompanied by the definition of its symbols and by the name of the function that realizes it in the MATLAB platform and in the Python reference. All processing takes place in the discrete-time complex baseband domain: the modulation symbols, the OFDM samples, the channel coefficients, and the noise samples are complex-valued sequences, which preserves all link-level behavior while avoiding RF carrier-level modeling [8].

3.1 Per-resource-element mimo model

On subcarrier k and OFDM symbol m, the received vector of the precoded MIMO-OFDM link is

$$y[k,m] = H[k,m]\, W[k,m]\, s[k,m] + n[k,m] \tag{1}$$

where s[k,m] denotes the L×1 vector of layer symbols, one QAM symbol per spatial layer; W[k,m] denotes the N_T×L digital precoding matrix that maps the L layers onto the N_T transmit antennas; H[k,m] denotes the N_R×N_T channel matrix whose entry (r, t) is the complex gain from transmit antenna t to receive antenna r; and n[k,m] denotes the N_R×1 receiver-noise vector. Equation (1) is evaluated once per resource element by apply_mimo_channel.m and mimo_channel.py. The validity of this per-subcarrier form is established by the OFDM processing of Section 3.2, and the construction of each factor of equation (1) is defined in Sections 3.3 to 3.5.

3.2 OFDM processing and the per-subcarrier channel

The transmitter generates the time-domain samples of each OFDM symbol with the unitary inverse discrete Fourier transform of the frequency-domain grid X[k],

$$x[n] = (1/\sqrt{N}) \sum_{k=0}^{N-1} X[k]\, e^{j2\pi kn/N}, \quad n = 0, 1, \ldots, N-1 \tag{2}$$

and the receiver recovers the subcarrier samples with the matching unitary forward transform,

$$Y[k] = (1/\sqrt{N}) \sum_{n=0}^{N-1} y[n]\, e^{-j2\pi kn/N} \tag{3}$$

Both transforms carry the same $1/\sqrt{N}$ scaling, so the pair preserves signal power exactly; ofdm_modulate.m, ofdm_demodulate.m, and ofdm.py apply this scaling explicitly, and the noise calibration of Section 3.3 relies on it. A cyclic prefix of N_{CP} samples, the copy of the last N_{CP} samples of each symbol,

$$x_{CP}[n] = x[\,(n - N_{CP}) \bmod N\,], \quad n = 0, 1, \ldots, N + N_{CP} - 1 \tag{4}$$

is prepended to every OFDM symbol. In a multipath channel the received waveform is the linear convolution of the transmitted waveform with the channel impulse response; when N_{CP} equals or exceeds the maximum channel delay spread, the cyclic prefix converts this linear convolution into a circular convolution over each symbol, and a circular convolution in the time domain corresponds to a per-subcarrier multiplication in the

frequency domain. After cyclic-prefix removal and the transform of equation (3), each subcarrier therefore obeys the flat scalar relation

$$Y[k] = H[k]\,X[k] + N[k] \tag{5}$$

where H[k] is the channel frequency response on subcarrier k and N[k] the frequency-domain noise sample. Equation (5) reduces one wideband frequency-selective channel to N independent flat subchannels, and every estimator and equalizer of Section 4 operates subcarrier by subcarrier on this basis. In the multi-antenna case the scalar quantities of equation (5) are replaced by the vector and matrix quantities of Section 3.1, which yields equation (1). The report, the MATLAB implementation, and the Python implementation all use this per-subcarrier representation as the common processing domain: the transmitter functions construct the frequency-domain resource grid, the channel functions act on it per subcarrier, and the OFDM functions are exercised separately by the waveform-domain measurements of Section 8.9.

3.3 Power and snr normalization

One convention governs every SNR-dependent result of the paper. The QAM constellations of all supported orders are scaled to unit average symbol power per layer, the precoder columns are orthonormal, and the total transmitted power consequently equals the layer count:

$$E\{\,|a|^2\,\} = 1\,, \quad W^H W = I_L\,, \quad E\{\,\|W\,s\|^2\,\} = L \tag{6}$$

The unit-power scaling is applied at the constellation level by qam_modulate.m and qam.py, and the orthonormal columns are returned by compute_precoder.m and compute_precoder.py. The SNR is defined as the total transmit SNR ρ, so the per-receive-antenna noise variance equals the total transmitted power L divided by the linear SNR:

$$n[k,m] \;\; CN(\,0,\,\sigma n\text{^}2\text{^}1\,)\,, \quad \sigma n\text{^}2\text{^} = L \cdot 10\text{^}{-SNR dB\sim/10\text{^}} \tag{7}$$

This is the statement implemented by the noise-variance assignment of apply_mimo_channel.m and mimo_channel.py. Because the noise variance is fixed per SNR point and independent of the channel realization, fading statistics appear correctly in the results, comparisons between antenna configurations are made at equal total transmit power, and beamforming gain appears as a genuine error-rate improvement rather than a hidden power increase. The 19.6 dB measurement of Section 8.4 is valid because equation (7) holds identically for both compared configurations. The capacity reference of equation (30) uses the matching equal-power allocation ρ/N_T across the transmit antennas.

3.4 The effective layer channel

The NR DM-RS pilots are precoded together with the data, so the receiver never observes, and never requires, the physical channel H alone. Every receiver algorithm of Section 4 operates on the effective layer channel

$$G[k,m] = H[k,m]\,W[k,m] \tag{8}$$

of dimension $N_R \times L$, the channel that the L transmitted layers experience. The channel estimator targets G, the equalizer inverts G, and true_effective_channel.m returns G in the ideal-CSI reference mode.

3.5 Precoding

Three precoder constructions are implemented. The per-subcarrier eigenbeamforming reference decomposes each channel matrix by the singular value decomposition,

$$H[k] = U[k]\,\Sigma[k]\,V^H[k] \tag{9}$$

and transmits along the first L columns of V[k], the L strongest spatial directions of that subcarrier. On frequency-selective channels this construction has a documented limitation: the singular vectors of neighboring subcarriers carry arbitrary independent phases, so the effective channel G[k] = H[k]W[k] loses its continuity in frequency. The DM-RS estimator of Section 4.1 interpolates between pilot subcarriers and therefore requires the estimated quantity to be smooth in frequency; a per-subcarrier SVD precoder violates this requirement. The default operating mode resolves the conflict with a wideband construction. The transmit covariance is accumulated over the K active subcarriers and diagonalized once per frame,

$$R = (1/K)\,\Sigma_k\, H^H[k]\,H[k]\,, \quad W = eig_L(R) \tag{10}$$

where $eig_L(\cdot)$ denotes the L dominant eigenvectors of the symmetrized covariance, returned as unit-norm precoder columns. One wideband matrix serves every subcarrier, so G[k,m] inherits the frequency smoothness of the physical channel, which is the condition the interpolating LS estimator requires; on a flat channel the wideband solution coincides with the per-subcarrier SVD. This pilot-compatible construction is the executed design of compute_precoder.m and compute_precoder.py, and it is the configuration under which all end-to-end results of Section 8 were produced. For single-layer transmission to one receive antenna, the maximum-ratio beamformer

$$w[k] = h^H[k] \,/\, ||h[k]|| \tag{11}$$

aligns the transmit phases so that the antenna contributions add constructively, and an unprecoded identity mode provides the spatial-multiplexing reference against which the massive-array gain of Section 8.4 is measured.

4. Receiver algorithms: derivations, procedures, and traceability

Each receiver algorithm is documented in three aligned forms: a formal derivation in which every symbol is defined, a boxed numbered procedure accompanied by a description of its flow, and a traceability entry in Table 2 that names the MATLAB function, the Python module, and the Simulink wrapper implementing it. The complete end-to-end procedure and its workflow diagram close the section.

4.1 DM-RS-based effective-channel estimation

On a DM-RS resource element the transmitted symbol r[k,m] is known to the receiver, so the observation model of equation (1) reduces, per layer, to Y = G r + N. Division of the received pilot sample by the known reference symbol yields the least-squares estimate at every pilot position,

$$\hat{G}_{LS}[k,m] = Y_{DMRS}[k,m] / r[k,m] \quad (12)$$

Substitution of the observation model into equation (12) gives the exact composition of the estimate,

$$\hat{G}_{LS}[k,m] = G[k,m] + N[k,m] / r[k,m] \quad (13)$$

which shows that the LS error consists purely of scaled noise: the estimator is unbiased, and its error variance equals σ_n^2 / $|r|^2$, which reduces to σ_n^2 for unit-power pilots. Equation (13) also determines the achievable estimation accuracy: an unsmoothed LS estimator tracks the noise floor at one decade per ten decibels of SNR, and the measured NMSE results of Section 8, Table 4 and Figure 13, follow this prediction over the full swept range. The slot carries two DM-RS symbols at positions m_1 and m_2, and because their noise samples are statistically independent, the implementation averages the two pilot observations,

$$\hat{G}[k] = (\hat{G}_{LS}[k,m_1] + \hat{G}_{LS}[k,m_2]) / 2 \quad (14)$$

which halves the pilot-noise variance, a 3 dB improvement obtained from pilot symbols that are already present in every slot. The pilot-position estimates are then extended to all allocated data resource elements by interpolation along frequency,

$$\hat{G}[k,m] = Interp\{ \hat{G}[k_{DMRS}] \} \quad (15)$$

performed on the real and imaginary parts separately, with constant (nearest-pilot) extension on the subcarriers outside the span of the outermost pilots; Section 6 documents the executed selection of this edge rule. The validity of equation (15) rests on the frequency smoothness of the effective channel, which is the property the wideband precoder of equation (10) preserves. Equations (12) to (15) constitute the executed estimator of every estimated-CSI result in this paper, implemented in estimate_effective_channel_ls.m and estimate_ls.py. A model-aware linear MMSE (Wiener) smoother on the same pilot pattern,

$$\hat{G}_{MMSE} = R_G (R_G + \sigma_n^2 I)^{-1} \hat{G}_{LS} \quad (16)$$

where R_G denotes the channel correlation across the pilot subcarriers [22], is a defined extension whose mean-square error is never worse than that of LS; it is specified in the platform design, it is not part of any executed campaign reported here, and no result figure or caption of this paper claims it. The terminology is applied consistently throughout the paper: MMSE equalization, Section 4.2, is a detector design and is not MMSE channel estimation. An ideal-CSI reference mode passes the true G[k,m] of equation (8) to the equalizer, and the difference between the estimated-CSI and ideal-CSI

modes isolates the channel-estimation loss from the equalizer and decoder behavior, a separation used throughout Section 8.

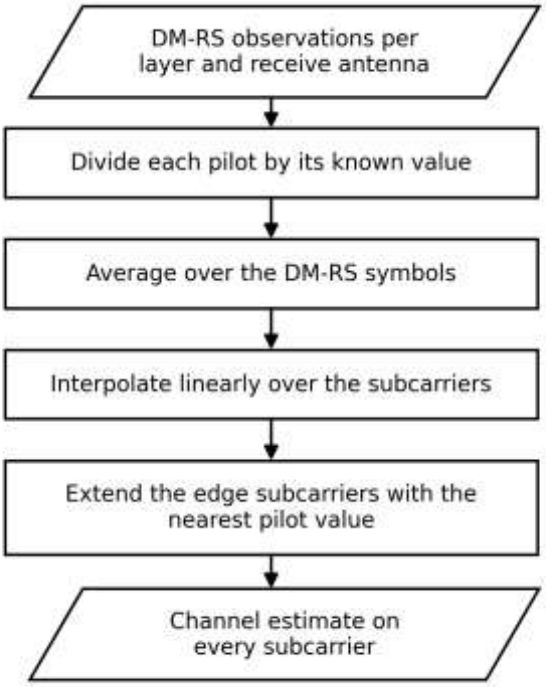

Figure 1. DM-RS-based effective-channel estimation pipeline, equations (12)-(15): LS pilot division, two-symbol averaging, and frequency interpolation (estimate_effective_channel_ls.m / estimate_ls.py / sl_estimator).

Algorithm 1: DM-RS-based effective-channel estimation
1: Input: received grid Y[k,m], DM-RS positions, known pilot symbols r, noise variance σn^2.
2: Extract the received DM-RS resource elements from the grid.
3: Form the LS estimate at every pilot position by the division of equation (12).
4: Average the two DM-RS symbol estimates according to equation (14).
5: Interpolate over frequency to every allocated data resource element, equation (15).
6: Output: estimated effective channel $\hat{G}$[k,m] for the equalizer.
Implementation: MATLAB estimate_effective_channel_ls.m · Python estimate_ls.py · Simulink sl_estimator.

The procedure operates as follows. It starts with the received resource grid and the known DM-RS layout of the frame. The pilot resource elements are first extracted from the grid, and the least-squares division of equation (12) is applied at every extracted position, producing one raw channel estimate per pilot subcarrier and per DM-RS symbol. The two DM-RS symbol estimates of each subcarrier are then averaged according to equation (14), which reduces the pilot-noise variance by half. The procedure concludes with the frequency interpolation of equation (15), which extends the averaged pilot-position estimates to every allocated data resource element and delivers the channel estimate consumed by the equalizer of Section 4.2.

4.2 Linear equalization: ZF and unbiased MMSE

Both equalizers form the layer estimate $\hat{s} = F\,y$ on each subcarrier from the estimated effective channel, and both are derived from an explicit optimality criterion. The zero-forcing equalizer minimizes the squared residual between the observation and its reconstruction,

$$J_{ZF}(s) = \| y - \hat{G}\, s \|^2 \tag{17}$$

Setting the gradient of equation (17) with respect to s to zero yields the normal equations

$$\hat{G}^H \hat{G}\, \hat{s} = \hat{G}^H y \tag{18}$$

whose solution defines the ZF equalization matrix

$$F_{ZF} = (\hat{G}^H \hat{G})^{-1} \hat{G}^H \tag{19}$$

The inverse exists whenever $N_R \geq L$ and the columns of $\hat{G}$ are linearly independent. Zero forcing removes inter-layer interference completely, but the criterion of equation (17) contains no noise term; whenever the channel matrix has small singular values, the inversion amplifies the noise strongly. The MMSE equalizer replaces the residual criterion with the mean-square estimation error,

$$J_{MMSE}(F) = E\{ \| s - F\, y \|^2 \} \tag{20}$$

Application of the orthogonality principle under the unit-power layers of equation (6) and the noise model of equation (7), followed by the matrix-inversion identity, gives the computationally efficient form that inverts an L×L matrix rather than an $N_R \times N_R$ matrix [10]:

$$F_{MMSE} = (\hat{G}^H \hat{G} + \sigma_n^2\, I)^{-1} \hat{G}^H \tag{21}$$

The noise-variance term regularizes the inversion: at high SNR the term vanishes and the MMSE solution converges to the ZF solution, and at low SNR the term dominates and the MMSE solution approaches a matched filter. The raw MMSE output is amplitude-biased toward the origin, which shifts hard-decision boundaries for amplitude-bearing constellations; division of each layer output by the corresponding diagonal element of the composite response,

$$\hat{s}_l \leftarrow [\, F\, y \,]_l \,/\, [\, F\, \hat{G} \,]_{ll} \tag{22}$$

removes the bias and produces the unbiased-MMSE output consumed by the demodulator. This normalization is the documented design of equalize_mimo.m and equalize.py, and the results of Section 8 accordingly label the executed detector unbiased MMSE. Because the channel is quasi-static over one slot, one equalizer matrix is computed per subcarrier and reused for all data resource elements of that subcarrier.

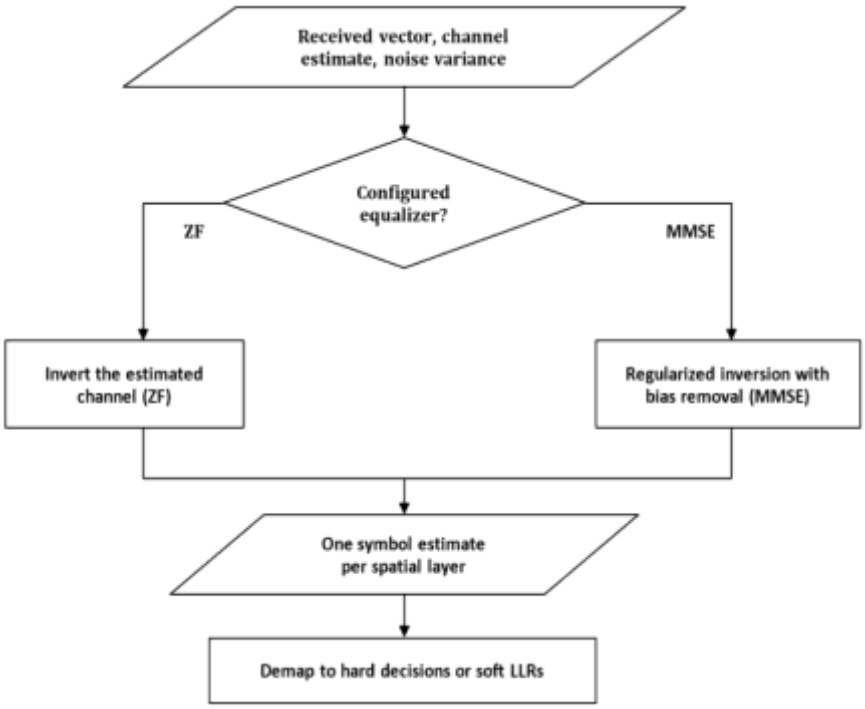

Figure 2. Linear MIMO equalization stage with the ZF and unbiased-MMSE options, equations (17)-(22) (equalize_mimo.m / equalize.py / sl_equalizer).

Algorithm 2: Equalization, detection, and metric extraction
1: Input: received grid Y[k,m], channel estimate $\hat{G}$ from Algorithm 1 (or true G in ideal-CSI mode), noise variance σn^2.
2: Per subcarrier, compute the equalizer matrix: ZF by equations (17)-(19) or MMSE by equations (20)-(21).
3: Form the layer estimates $\hat{s} = F\,y$ for every data resource element.
4: Apply the unbiased-MMSE diagonal normalization of equation (22).
5: Demap each layer symbol: hard decision in the uncoded study; max-log LLR values, equation (23), in the coded study.
6: Descramble; in the coded study, recover the rate matching and execute the TS 38.212 LDPC decoder.
7: Accept the transport block if the CRC check passes; record the block decision.
8: Update BER, BLER, EVM, NMSE, SINR, throughput, and capacity, equations (27)-(30).
Implementation: MATLAB equalize_mimo.m, qam_demodulate_hard.m, check_crc24a.m, compute_frame_metrics.m · Python equalize.py, qam.py, crc24a.py, metrics.py · Simulink sl_equalizer, sl_metrics.

The procedure operates as follows. It starts with the received resource grid, the channel estimate produced by Algorithm 1, and the known noise variance of equation (7). For each subcarrier the equalizer matrix is first computed, by equation (19) in ZF mode or equation (21) in MMSE mode, and the layer estimates are then formed for every data resource element of that subcarrier. The unbiased normalization of equation (22) is subsequently applied to each layer. The equalized symbols are then demapped, by hard decision in the uncoded study or by the max-log rule of equation (23) in the coded study,

the bit stream is descrambled, and, in the coded study, the rate matching is recovered and the LDPC decoder is executed. The procedure concludes with the CRC decision on the recovered transport block and the update of the frame-level metrics of equations (27) to (30).

4.3 Soft demodulation and detection quality

In the coded study each equalized layer symbol z is demapped into soft bits with the max-log approximation

$$LLR_i(z) \approx (1/\sigma_{eff}^2) \left[\min_{a \in S_i^0} |z - a|^2 - \min_{a \in S_i^1} |z - a|^2 \right] \tag{23}$$

where S_i^0 and S_i^1 denote the constellation subsets whose bit i equals zero and one, respectively; the sign convention, positive LLR for bit zero, is checked by a dedicated verification gate before every coded campaign. The scaling of equation (23) requires the effective post-equalization noise variance of the layer, and this variance has an exact expression. The layer error decomposes into a residual-interference component and a filtered-noise component,

$$e = \hat{s} - s = (F \hat{G} - I) s + F n \tag{24}$$

and the effective noise variance of layer l equals the power of the corresponding error row,

$$\sigma_{eff,l}^2 = || (F \hat{G} - I)_l ||^2 + \sigma_n^2 || F_l ||^2 \tag{25}$$

The same decomposition gives the post-equalization SINR of layer l,

$$SINR_l = | [F \hat{G}]_{ll} |^2 / (\Sigma_{j \neq l} | [F \hat{G}]_{lj} |^2 + \sigma_n^2 || F_l ||^2) \tag{26}$$

Under ideal CSI with ZF equalization the interference term of equation (25) vanishes, and under estimated CSI it grows with the channel-estimation error of equation (13). Equations (13) and (25) together establish, prior to any simulation, that estimated-CSI curves must lie to the right of ideal-CSI curves, and the measured displacement in Section 8.2 quantifies this penalty for the executed configuration.

4.4 Performance metrics

Every metric reported in Section 8 is defined here. Link reliability is measured by the bit and block error rates,

$$BER = N_{bit,error} / N_{bit,total} , \quad BLER = N_{block,error} / N_{block,total} \tag{27}$$

where the block decision is the CRC verdict, so the BLER is a decoded-transport-block statistic rather than a symbol statistic. Modulation accuracy is measured by the RMS error vector magnitude between the equalized and the transmitted symbols,

$$EVM_{RMS} = \sqrt{ (\Sigma_q |\hat{s}[q] - s[q]|^2 / \Sigma_q |s[q]|^2)} \tag{28}$$

Estimation accuracy is measured by the normalized mean-square error of the channel estimate with respect to the true effective channel,

$$NMSE = E\{ || \hat{G} - G ||^2 \} / E\{ || G ||^2 \} \tag{29}$$

and useful delivery is measured by the throughput of CRC-passed information bits over the simulated time, normalized by the occupied bandwidth B into the spectral efficiency η_{SE} = Throughput / B. The information-theoretic reference is the per-subcarrier MIMO capacity under equal power allocation [7],

$$C[k] = \log_2 \det(I_{N_R} + (\rho/N_T)\, H[k]\, H^H[k]) \quad (30)$$

which is reported as a reference bound and never interpreted as achieved coded throughput. All of equations (27) to (30) are computed per frame by compute_frame_metrics.m and metrics.py from identical symbol definitions, which is a requirement for the cross-implementation comparison of Section 6.

4.5 End-to-end procedure and workflow

Algorithm 3 states the complete end-to-end procedure in the order in which all three environments execute it, and Figure 3 presents the same procedure as a workflow diagram. Every box of the diagram carries three lines: the processing step, its governing equations, and the implementing MATLAB function, Python module, and Simulink block. The diagram therefore documents the complete correspondence between the algorithms and their implementations: every processing step of the diagram is traceable to the equations of this section and to a specific MATLAB function, Python module, and Simulink block, and each step can be verified directly against its source code in either language.

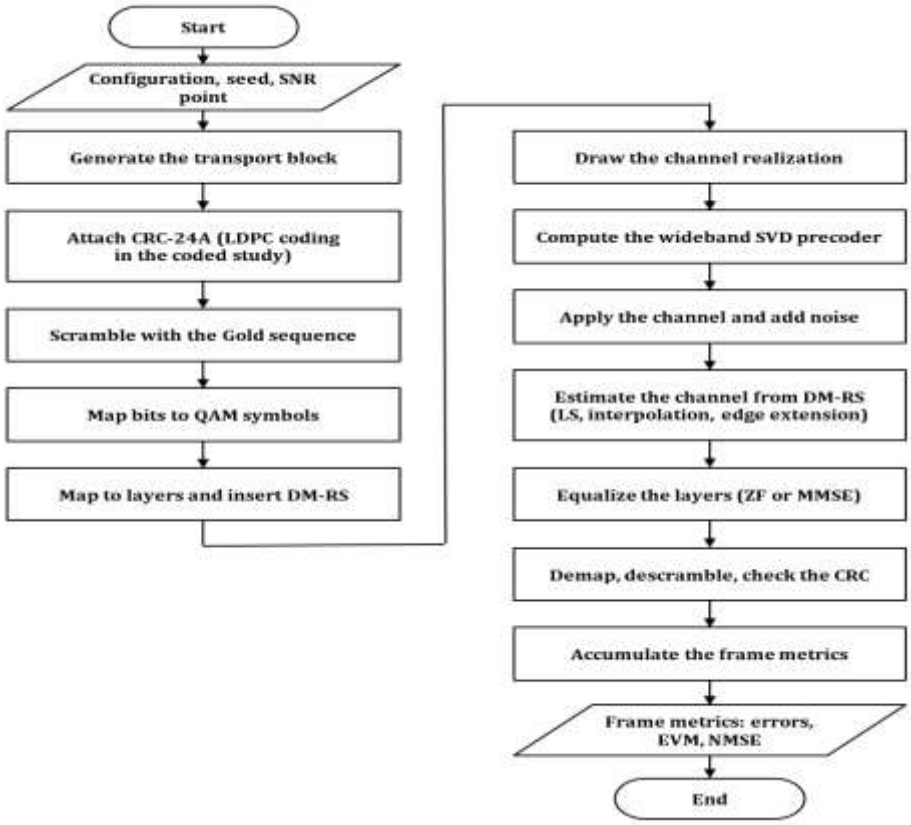

Figure 3. End-to-end algorithm workflow of one Monte Carlo trial, grouped into the transmitter, channel, receiver, and metric stages; the implementing MATLAB, Python, and Simulink functions of every step are listed in Table 2.

Algorithm 3: End-to-end PDSCH link-level simulation (one Monte Carlo trial)

1: Generate the transport block b of A information bits (build_frame).

2: Attach the CRC-24A; in the coded study, apply TS 38.212 LDPC encoding and rate matching (append_crc24a / nrDLSCH).

3: Scramble the coded bits with the Gold sequence (scramble_bits, nr_gold_sequence).

4: Map groups of bits to unit-power QAM symbols, equation (6) (qam_modulate / qam.py).

5: Arrange the symbols into L layers and insert the DM-RS pilots into the resource grid (build_frame).

6: Compute the wideband eigenbeamforming precoder W, equations (9)-(11) (compute_precoder).

7: Draw one independent channel realization H (generate_channel).

8: Apply the effective channel and receiver noise, $y = HWs + n$ with $\sigma n^2 = L \cdot 10^{\wedge}(-SNR/10)$, equations (1) and (7) (apply_mimo_channel / mimo_channel.py).

9: Estimate the effective channel from the DM-RS, Algorithm 1, equations (12)-(15), or use the true G in ideal-CSI mode.

10: Equalize, detect, decode, and check the CRC, Algorithm 2, equations (17)-(23).

11: Accumulate the metrics of equations (27)-(30) (compute_frame_metrics / metrics.py).

The trial proceeds as follows. It starts with the generation of one transport block of information bits, to which the CRC-24A is attached; in the coded study the protected block additionally passes through TS 38.212 LDPC encoding and rate matching. The coded bit stream is then scrambled with the Gold sequence and mapped to unit-power QAM symbols, and the symbols are arranged into the configured spatial layers and placed, together with the DM-RS pilots, into the resource grid. The wideband eigenbeamforming precoder is then computed from the channel realization of the trial, the precoded grid passes through the effective channel, and receiver noise of the variance defined by equation (7) is added. On the receiver side, the effective channel is estimated from the DM-RS by Algorithm 1, the data resource elements are equalized and detected by Algorithm 2, the coded study decodes the LDPC code, and the CRC verdict on the recovered transport block concludes the receiver processing of the trial. The trial ends with the accumulation of the frame metrics, and the Monte Carlo campaign repeats the trial with independent random draws until the configured frame count of the SNR point is reached.

Table 2 completes the alignment between the mathematics and the three implementations. Each row of the table lists one processing block of the workflow together with its governing equations; the MATLAB column names the executed function files of the primary platform, the Python column names the modules of the uncoded reference implementation, and the Simulink column names the wrapper block of the testbench. The Simulink wrappers are thin MATLAB Function blocks around the verified project functions, so the testbench executes the identical numerical code, and the LDPC row is MATLAB-only in accordance with the stated scope of the Python reference.

Table 2. Algorithm-to-implementation traceability across the three execution environments.

Processing block	**Equations**	**MATLAB**	**Python**	**Simulink**
Transport block, CRC, scrambling, QAM, layers, DM-RS grid	(6)	build_frame.m, append_crc24a.m, scramble_bits.m, qam_modulate.m, nr_gold_sequence.m	build_frame.py, crc24a.py, scramble_bits.py, qam.py, nr_gold_sequence.py	sl_transmitter
OFDM modulation / demodulation	(2)-(5)	ofdm_modulate.m, ofdm_demodulate.m	ofdm.py	sl_waveform (monitoring branch)
Channel generation (AWGN / Rayleigh / Rician / TDL)	Sec. 3.2	generate_channel.m	generate_channel.py	sl_channel
Wideband eigenbeamforming precoder	(9)-(11)	compute_precoder.m	compute_precoder.py	sl_precoder
Effective channel and noise application	(1), (7)	apply_mimo_channel.m	mimo_channel.py	sl_apply_channel
LS effective-channel estimation	(12)-(15)	estimate_effective_channel_ls.m	estimate_ls.py	sl_estimator
ZF / unbiased-MMSE equalization	(17)-(22)	equalize_mimo.m	equalize.py	sl_equalizer
Demapping, CRC decision, metrics	(23), (27)-(30)	qam_demodulate_hard.m, check_crc24a.m, compute_frame_metrics.m	qam.py, crc24a.py, metrics.py	sl_metrics
LDPC coded transport chain (TS 38.212)	Sec. 8.7	run_ldpc_campaign.m via 5G Toolbox nrDLSCH	not implemented (uncoded reference)	not in the testbench

5. Simulator Architecture and Execution Environments

This section illustrates the simulator architecture: the three execution environments and their scopes, the end-to-end signal path, and the Simulink testbench.

5.1 Three execution environments and their scopes

The simulator exists as three executions of one algorithmic specification, and their scopes are stated explicitly because the strength of their agreement as verification evidence depends on them. The MATLAB implementation is the primary platform and implements the complete coded and uncoded study, including the TS 38.212 LDPC transport chain built on the 5G Toolbox DL-SCH components [12]. The Python implementation, written independently on NumPy and SciPy [13], is an uncoded, CRC-protected reference model: it independently realizes the MIMO-OFDM processing, the Gold-sequence scrambling, the DM-RS construction, the LS effective-channel estimation, the ZF and unbiased-MMSE detection, and the complete metric chain, and it is the codebase against which the uncoded MATLAB campaigns are cross-verified. It does not reproduce the MATLAB LDPC chain, and no coded result of this paper claims Python confirmation. The Simulink testbench executes the same verified MATLAB functions under Simulink scheduling and instrumentation. The MATLAB and Python codebases share no source code; they share only the mathematical specification of Sections 3 and 4, and their agreement therefore constitutes independent verification evidence.

5.2 End-to-end signal path

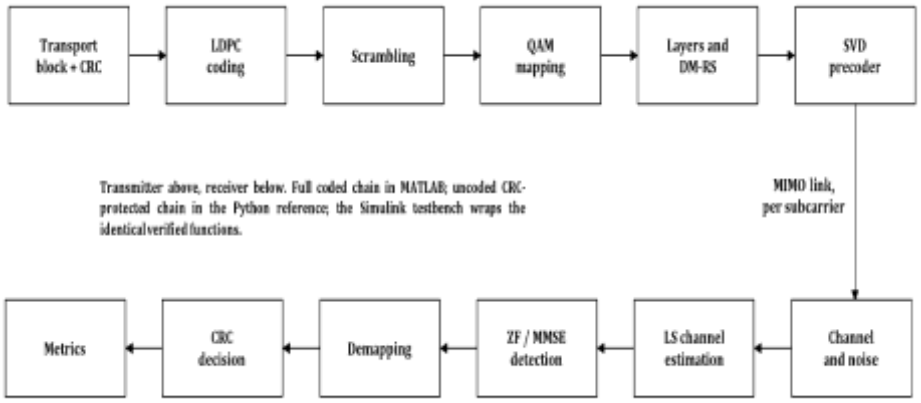

Figure 4. End-to-end PDSCH link-level system model. The full coded chain is executed in MATLAB; the Python reference implements the uncoded, CRC-protected chain.

Figure 4 shows the complete signal path of one Monte Carlo realization; every block corresponds to one equation of Sections 3 and 4 and to one row of Table 2.

5.3 Simulink testbench

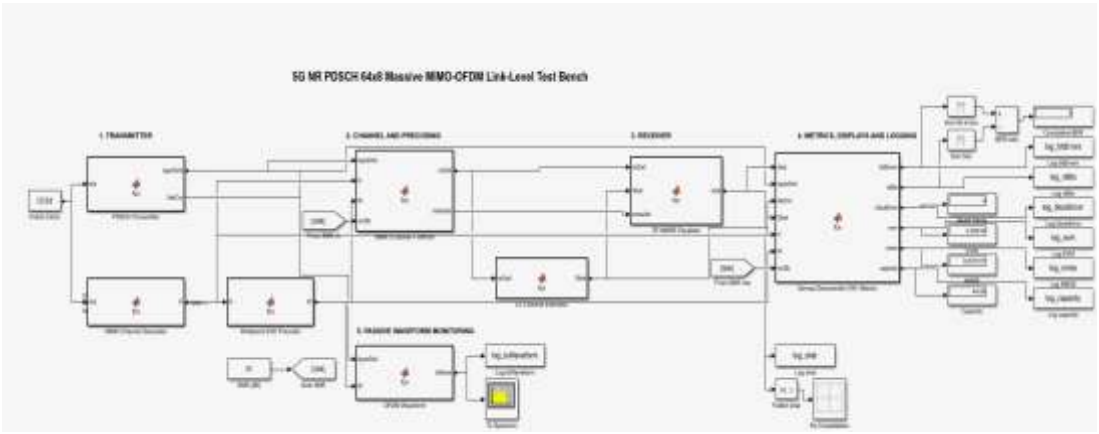

Figure 5. Simulink testbench of the verified chain, 64×8 massive profile, executing one Monte Carlo frame per simulation step.

Figure 5 shows the Simulink testbench built for the 64×8 profile, and its operation proceeds in three phases. In the initialization phase, the model initialization callback loads the locked massive-profile configuration, verifies the 64×8 antenna and four-layer dimensions against the model interfaces, seeds the random stream, and executes the verification gates of Section 6; a gate failure stops the model before the first simulation step, so the testbench cannot produce an unverified result. In the execution phase, the model advances one complete Monte Carlo frame per simulation step: sl_transmitter generates the transport block and the resource grid, sl_channel draws the channel realization, sl_precoder computes the wideband eigenbeamformer, sl_apply_channel applies the effective channel and the calibrated noise of equation (7), sl_estimator and sl_equalizer execute the receiver algorithms of Section 4, and sl_metrics accumulates the frame metrics. A passive monitoring branch taps the transmitted CP-OFDM waveform into a Spectrum Analyzer instrument and the equalized layer symbols into a Constellation Diagram instrument; the branch consumes no random numbers and therefore changes no metric result, and its logged waveforms provide the measurements of Section 8.9. In the evaluation phase, a sweep driver repeats the run over the full SNR grid, aggregates the per-point metrics, writes them to the result file, and compares every point against the reference campaign under the tolerances of Section 6; the outcome of this comparison is reported in Sections 8.6 and 8.8.

6. Verification Methodology

This section states the verification methodology: the unit-level gates, the closed-form anchoring, the cross-implementation tolerance protocol with its regression re-execution, and one documented defect-detection case.

6.1 Verification principle and gates

The methodology is based on one rule: a numerical curve is accepted only if it was produced by executing the simulator, passed the verification gates, and was saved with full file-level traceability. Formally, every accepted result is a function of five stored artifacts,

$$R_{final} = f(C_{sim}, S, X_{code}, D_{CSV}, V_{gate}) \tag{31}$$

where C_{sim} denotes the locked configuration, S the seed set, X_{code} the executed source, D_{CSV} the saved numerical output, and V_{gate} the passed acceptance record. Every figure in Section 8 is regenerable from its stored output file and the simulation source alone.

Four unit-level gates run before any campaign. The constellation-power gate verifies that equation (6) holds exactly over the full constellation set of every supported order. The OFDM round-trip gate measures the relative reconstruction error of the transform pair,

$$\varepsilon = || x - IFFT(FFT(x)) || / || x || \tag{32}$$

and records $\varepsilon = 1.19\times10^{-15}$, at double-precision machine level, confirming the unitary scaling of equations (2) and (3). The CRC gate verifies that valid blocks are accepted and corrupted blocks rejected, and the noiseless modulation round trip is error-free for QPSK, 16-QAM, 64-QAM, and 256-QAM. Three additional coded-chain gates run before either LDPC campaign: an error-free noiseless coded round trip with CRC pass, a check of the LLR sign convention of equation (23), and a full coded decode at 30 dB. All gate scripts abort the campaign on any failure.

6.2 Anchoring against closed-form references

End-to-end behavior is verified against closed-form theory. The simulated QPSK BER in AWGN lies on the closed form with deviations of 0.007, 0.005, and 0.007 decades at 4, 6, and 8 dB under ideal CSI, and the flat-Rayleigh QPSK chain tracks the exact fading expression with the diversity-one slope; an exact identity-channel capacity check agrees with the closed form to every printed digit. Under LS estimation the same AWGN deviations grow to 0.151, 0.206, and 0.292 decades, which is not a defect but the measured pilot-noise penalty of equation (13), and the consistency of this penalty across independent tests is itself verification evidence.

6.3 Cross-implementation tolerance protocol

Cross-implementation verification then compares the MATLAB and Python executions of the same locked uncoded configuration under explicit tolerances. The comparison is statistical and metric-level: the two codebases consume independent random streams,

and their per-SNR aggregate metrics are compared, which verifies the complete processing and metric chain at the level Monte Carlo statistics permit. It is not a sample-by-sample equivalence test; a common-input checkpoint protocol, in which both implementations consume identical stored bits, channels, noise, and pilots and are compared at the waveform, grid, and symbol level, is demonstrated once in the defect investigation below and its systematic integration across the chain remains future work in Section 10. Error-rate quantities are compared on a logarithmic decade metric,

$$\Delta_{dec} = | \log_{10}(x_{MATLAB}) - \log_{10}(x_{Python}) | \quad (33)$$

because a fixed decade tolerance treats a deviation at 10^{-1} and at 10^{-4} with equal statistical strictness, while smooth quantities are compared on relative tolerances and the coarsely quantized BLER on an absolute tolerance. The tolerances are set from the Monte Carlo resolution of the executed frame counts: BER within 0.3 decades, NMSE within 0.15 decades, EVM within 10% relative, capacity within 2% relative, and BLER within 0.25 absolute. Algorithm 4 states the protocol, and Figure 6 presents it in the same workflow form as the end-to-end diagram of Figure 3, including the acceptance decision and the defect-handling path.

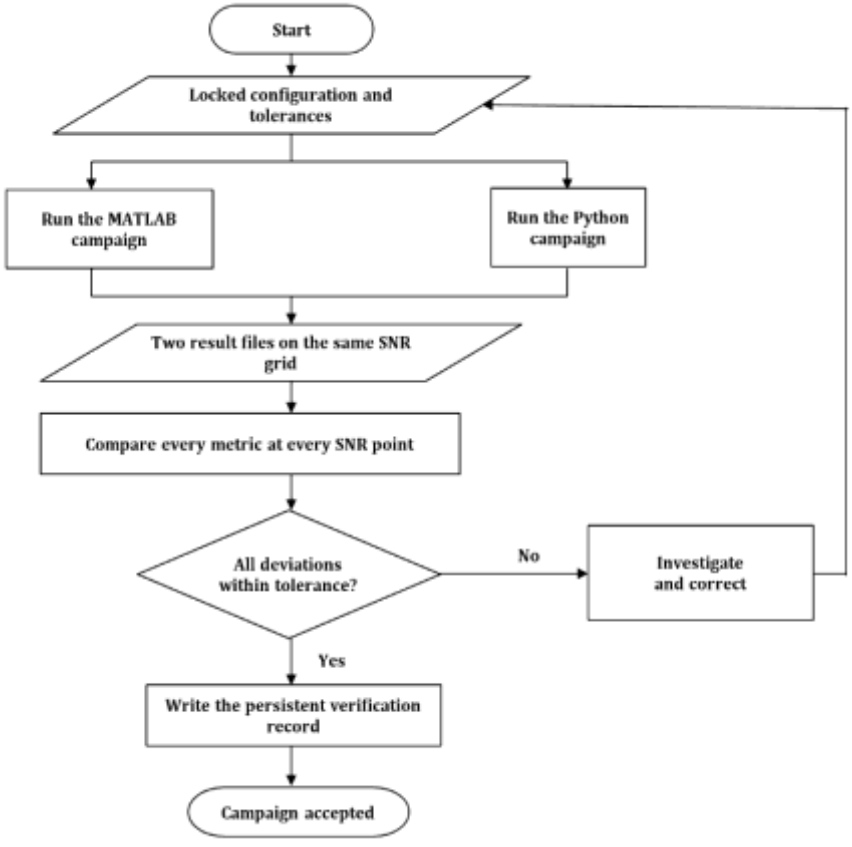

Figure 6. Cross-verification protocol workflow, Algorithm 4: locked configuration, independent MATLAB and Python campaigns, per-metric tolerance evaluation, persistent record, and the acceptance decision with its defect-handling path.

Algorithm 4: MATLAB-Python cross-verification protocol
1: Lock one configuration C_{sim} and its seed set; export it to both implementations (config.m / config.py, sim_profile.txt).
2: Execute the MATLAB campaign; store the per-SNR results and the configuration log.
3: Execute the Python campaign on an independent random stream; store the results with the recorded bit and block error counts.
4: For every SNR point and every metric, compute the deviation under its metric class, equation (33), and compare it against its tolerance.
5: Record every comparison with its deviation, tolerance, and verdict in the persistent cross-verification record (cross_verify_csv.py).
6: Accept the campaign only if every comparison passes; on any failure, treat the discrepancy as a defect until an executed correction or a recorded statistical explanation exists, then re-execute steps 2 to 5.

The protocol operates as follows. It starts with the locking of one simulation configuration and its seed set, which are exported to both implementations so that neither codebase can drift from the compared specification. The MATLAB campaign is executed first and its per-SNR results are stored together with the configuration log; the Python campaign is then executed on an independent random stream and its results are stored together with the raw bit and block error counts. For every SNR point and every metric, the deviation between the two implementations is computed under the metric class of equation (33) and compared against the corresponding tolerance, and each comparison is written, with its deviation, tolerance, and verdict, into the persistent cross-verification record. The protocol concludes with the acceptance decision: the campaign is accepted only if every comparison passes, and any failure is treated as a defect that requires an executed correction, or a recorded statistical explanation, followed by re-execution, before acceptance is possible.

Regression testing completes the methodology: the same locked configuration is re-executed on independent random streams, in MATLAB twice and in the Simulink testbench, and the runs are compared under the same tolerances, so implementation changes cannot alter the results undetected.

6.4 A Documented defect-detection case

The methodology detected and resolved one genuine cross-implementation defect during the work reported here, and the case is documented because it demonstrates the protocol operating as designed. The tight-statistics adaptive campaigns of Section 8.6 exposed a factor-of-two BER discrepancy between the two implementations at the 10 dB operating point, 0.27 decades, a deviation that the coarse-tolerance record had legitimately passed. A paired experiment on 250 identical stored frames localized the cause to the interpolation edge rule of equation (15): the MATLAB implementation linearly extrapolated beyond the outermost pilots while the Python implementation applied constant extension, and the entire discrepancy resided in the six subcarriers outside the pilot span, with the interior estimation error identical to four significant digits and the

edge estimation error 70% larger under extrapolation, producing 2.12 times the bit errors on identical frames. Constant extension was adopted in both implementations: it is immune to edge pilot noise, and the small edge bias it introduces on frequency-selective channels is the province of the Wiener extension of equation (16). After the correction, the re-executed cross-verification record passes all thirty-five comparisons with the largest bit-error-rate deviation at 0.046 decades, and the equal-criteria adaptive campaigns of the two implementations agree within overlapping 95% confidence intervals at every measured point.

7. Statistical Qualification of Monte Carlo Estimates

This section states the statistical qualification applied to every reported estimate: exact confidence intervals, upper bounds for zero-error points, and the executed adaptive stopping rule.

7.1 Exact confidence intervals

Every BER and BLER value in this paper is a binomial proportion estimated from a finite executed sample [29]: k observed errors in N evaluated bits or blocks give the point estimate $\hat{p} = k/N$. A point estimate alone does not state how strongly the finite sample constrains the true error rate, so every reported estimate carries an exact two-sided Clopper-Pearson 95% confidence interval [27], computed from the recorded error counts stored with each campaign:

$$p_L = B(\alpha/2;\, k,\, N-k+1), \quad p_U = B(1-\alpha/2;\, k+1,\, N-k) \tag{34}$$

where B(q; a, b) denotes the q-quantile of the Beta(a, b) distribution and $\alpha = 0.05$. The Clopper-Pearson construction is exact rather than asymptotic, which is important at the extreme counts encountered in link-level evaluation: very large N with very small k on the bit statistic, and very small N on the block statistic.

7.2 Zero-error operating points as upper bounds

Zero observed errors require particular care. An observed count of zero does not establish an error rate of zero; it establishes an upper bound whose scale is set by the sample size. The exact bound follows from equation (34) with k = 0, and the rule of three [28] gives its close approximation,

$$p_U \approx 3 / N \tag{35}$$

at the 95% level. Throughout Section 8, operating points with zero recorded errors are therefore reported as upper confidence bounds, never as zeros. Zero bit errors in the 552,480,000 bits evaluated at one high-SNR point of the adaptive massive campaign support the statement that the true BER lies below 6.64×10^{-9} (exact) or approximately 5.43×10^{-9} (rule of three) at 95% confidence, and zero block errors in the corresponding 20,000 blocks bound the BLER below 1.9×10^{-4}.

7.3 Adaptive Monte Carlo stopping

The principal massive campaign of this paper is executed under adaptive Monte Carlo stopping: each SNR point runs until 500 bit errors or 100 block errors are accumulated, within a budget of at least 500 and at most 20,000 frames, and every result row stores the frame count, the evaluated bits, the raw error counts, the interval bounds, the stopping reason, the seed, and the wall-clock runtime. The compact campaign uses forty fixed frames per point and the fine coded campaign one hundred transport blocks per point, and the confidence intervals make the resulting resolution explicit at every operating point.

8. Executed Results and Discussion

All results were produced by executing the simulator under the provenance rule of equation (31); no illustrative or expected curve appears. The algorithms are NR-oriented, but the executed Monte Carlo allocation uses a 12-resource-block reduced grid, 144 active subcarriers on a 256-point FFT with 4.32 MHz occupied bandwidth, to bound the computational cost of the 64×8 study; no wideband throughput is computed or implied from this allocation, and every figure of this section is to be read at the stated grid. Table 3 states the end-to-end massive configuration exactly as recorded in the stored configuration log; the first column of the table names the configuration category, and the second column reproduces the executed value together with the equation that governs it.

Table 3. Executed end-to-end massive MIMO configuration.

Parameter	**Executed value**
SNR grid / trials	−10 to 20 dB in 5 dB steps; adaptive stopping with 500 to 20,000 frames per point (500 bit-error / 100 block-error targets)
Numerology	144 subcarriers (12 RB, reduced allocation), 14 symbols per 0.5 ms slot, 30 kHz SCS, FFT 256, normal CP
Antennas / layers	64×8, L = 4, wideband eigenbeamforming precoder, equation (10)
Modulation / transport	16-QAM, uncoded transport block with CRC-24, 27,624 payload bits per slot
DM-RS / estimation	Two pilot symbols per slot; LS estimation with two-symbol averaging and frequency interpolation, equations (12)-(15)
Channel	Flat i.i.d. Rayleigh, one independent realization per frame
Equalizer	Unbiased MMSE, equations (21)-(22)

Each massive-campaign SNR point evaluates between 13.8 million and 552.5 million information bits under the adaptive rule of Section 7. The compact campaign of Section 8.5 uses the 4×4, two-layer profile over the five-tap TDL channel with forty frames and 598,080 bits per point, and the coded campaigns of Section 8.7, executed in MATLAB only, run on the same compact TDL profile, where frequency selectivity makes channel coding consequential.

8.1 Anchoring to Closed-Form Theory

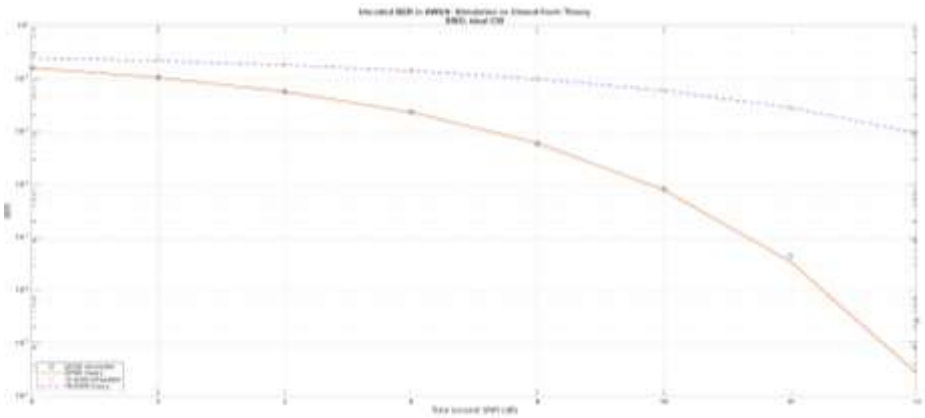

Figure 7. Uncoded BER in AWGN against closed-form theory: QPSK and 16-QAM, SISO, ideal CSI, 12-RB grid.

Figure 7 places the simulated AWGN points on the closed-form references across five decades of error rate, from above 10^{-1} to the 10^{-6} region. Every point with a meaningful error count lies on theory; the final QPSK point rises slightly above the line because only a small number of error events remain at that error rate under the executed trial count, which is the finite-sample behavior the intervals of Section 7 quantify. This comparison jointly validates the modulation mapping, the OFDM processing of equations (2)-(5), the noise calibration of equation (7), and the demapping.

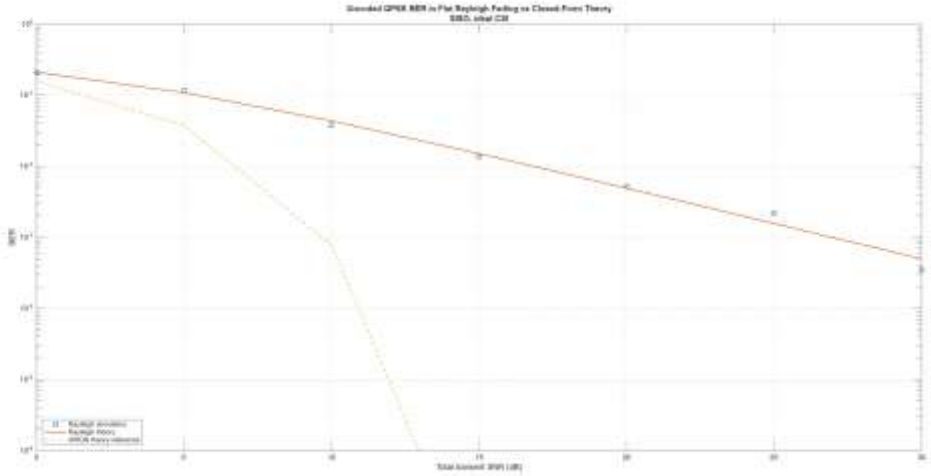

Figure 8. Uncoded QPSK BER in flat Rayleigh fading against closed-form theory, with the AWGN reference.

Figure 8 shows the flat-Rayleigh chain following the exact fading expression with the diversity-one slope of one decade per ten decibels. The highest swept points scatter around the theory line because the statistic there is carried by rare deep-fade events; the trend and slope remain those of the closed form.

8.2 Equalization and the Measured Cost of Channel Estimation

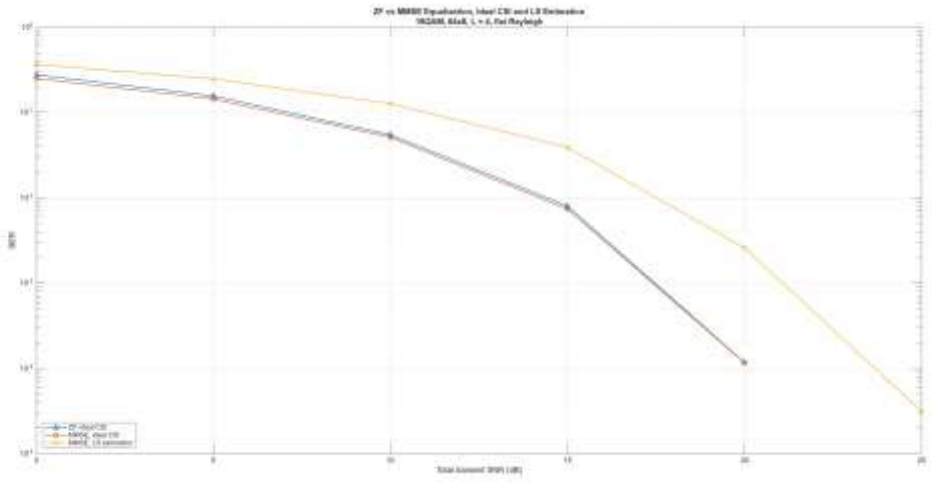

Figure 9. ZF and MMSE equalization with ideal CSI, and MMSE with LS-estimated CSI: identity (unprecoded) spatial multiplexing, 16-QAM, 64×8 grid, L = 4, flat Rayleigh, 12 RB.

The comparison of Figure 9 deliberately uses unprecoded transmission: under the eigenbeamforming precoder the effective Gram matrix is diagonal on the flat channel and ZF and MMSE coincide identically, so the equalizer contrast requires the general unprecoded channel. Two observations follow. With eight receive antennas against four layers the unprecoded effective channel is tall and well conditioned, so the ideal-CSI ZF and MMSE curves coincide over the entire sweep, at approximately 1.4×10^{-4} at 20 dB; the regularization of equation (21) provides no additional benefit in this regime. The curve driven by the LS estimate is displaced several decibels to the right at every operating point, at approximately 3.3×10^{-3} at 20 dB, because the pilot-noise-limited error of equation (13) adds to the thermal noise everywhere. At these array dimensions, estimation quality rather than equalizer choice sets the achievable error rate, a conclusion with direct consequences for pilot design and estimator selection.

8.3 Capacity Scaling

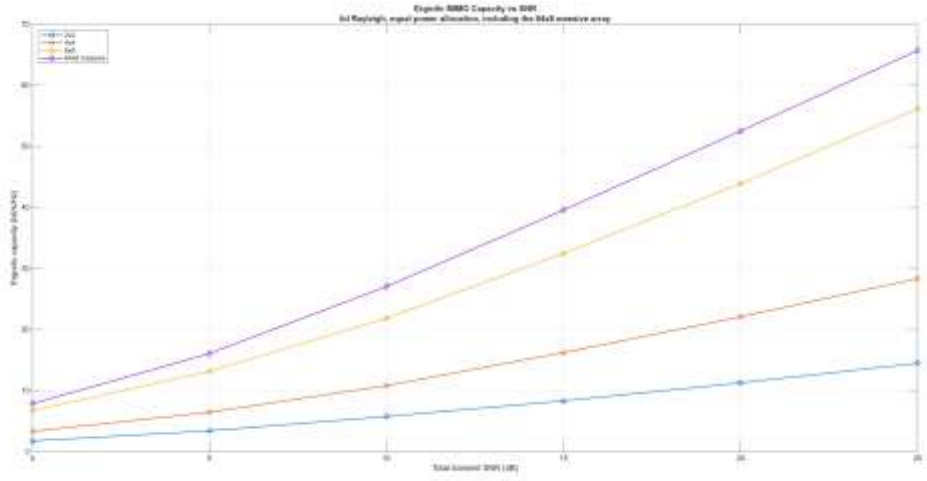

Figure 10. Ergodic MIMO capacity against SNR for 2×2, 4×4, and 8×8 i.i.d. Rayleigh channels and the 64×8 array, equal power allocation, equation (30).

At 25 dB the measured ergodic capacities are approximately 14, 28, and 56 bit/s/Hz for the square 2×2, 4×4, and 8×8 configurations, doubling with the antenna count as the $\min(N_T, N_R)$ multiplexing slope requires. The 64×8 curve lies above the 8×8 curve at every point, at approximately 66 bit/s/Hz at 25 dB, and shares its high-SNR slope: both configurations have minimum dimension eight, so the vertical offset is the array gain of the sixty-four-antenna transmitter.

8.4 Combined Array-Dimension and Eigenbeamforming Gain

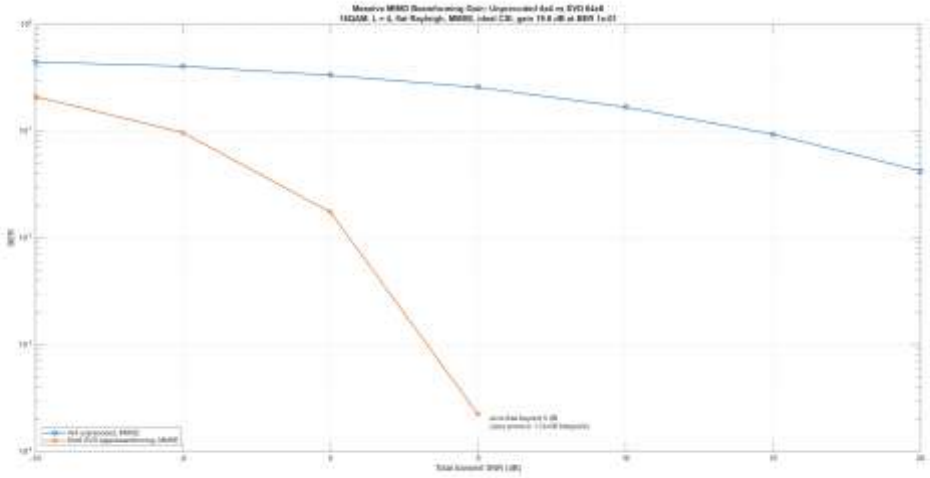

Figure 11. Combined array-dimension and eigenbeamforming gain under equal total transmit power: unprecoded 4×4 against eigenbeamformed 64×8, 16-QAM, L = 4, flat i.i.d. Rayleigh, MMSE, ideal effective CSI.

Figure 11 isolates the contribution of the massive configuration. Both systems carry the same four layers, the same modulation, the same fading statistics, and the same total transmit power per equation (6), and ideal CSI removes the estimation penalty of Section 8.2, so the comparison measures the array and its precoder alone. The comparison changes three quantities simultaneously, the transmit antennas from four to sixty-four, the receive antennas from four to eight, and identity precoding to wideband eigenbeamforming, so the measured separation is a combined array-dimension and eigenbeamforming gain, not a pure beamforming gain. The unprecoded 4×4 reference decreases slowly and remains above 4×10^{-2} at 20 dB, while the eigenbeamformed 64×8 system passes 1.7×10^{-2} at 0 dB, reaches approximately 2×10^{-4} at 5 dB, and records zero errors at every point above 5 dB, which by Section 7 bounds the true BER below 2.23×10^{-6} per point at 95% confidence. The measured horizontal separation at BER 10^{-1} is 19.6 dB. The result is stated with its applicability conditions and is not presented as a universal massive MIMO gain: it holds for the flat i.i.d. Rayleigh model without spatial correlation, four layers, ideal effective CSI, the configured 64×8 against 4×4 dimensions, and the selected BER operating point. The comparison was executed twice on independent random streams, inside the comparison campaign and as a dedicated run, and the two executions agree within Monte Carlo spread.

8.5 Compact-Profile End-to-End Campaign

The compact campaign exercises the complete estimated-CSI receiver chain on the 4×4, two-layer, 16-QAM profile over the five-tap TDL channel, with the SVD precoder, LS estimation, and MMSE equalization, at forty frames and 598,080 evaluated bits per SNR point over the grid 0 to 25 dB. This profile is the frequency-selective counterpart of the massive campaign: the TDL delay taps make the interpolation step of equation (15)

consequential, and the campaign therefore measures the full estimator of Section 4.1 under the conditions it was designed for.

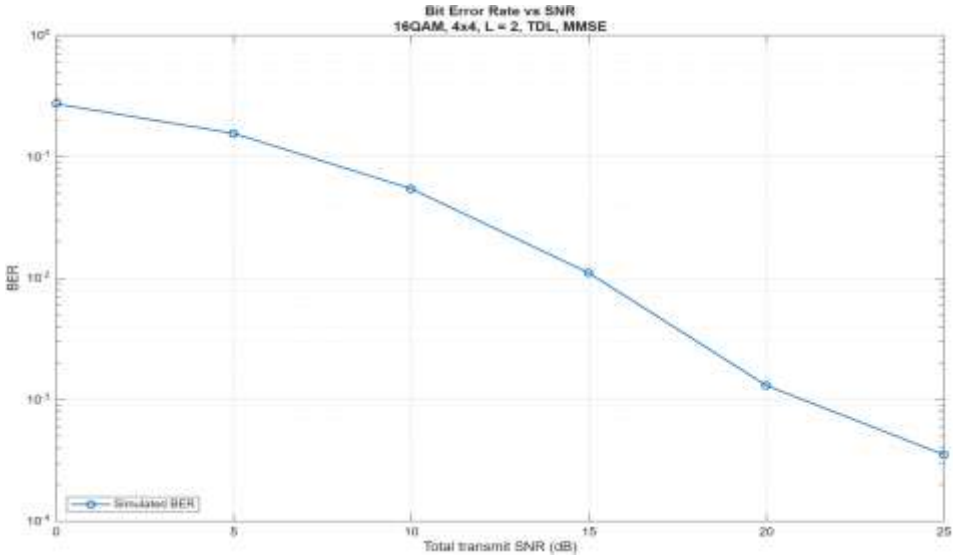

Figure 12. Uncoded BER against SNR, compact 4×4 profile, L = 2, 16-QAM, TDL channel, LS estimation, MMSE equalization. MATLAB campaign.

Figure 12 shows the measured BER falling from 2.73×10^{-1} at 0 dB to 3.53×10^{-4} at 25 dB. The uncoded BLER follows the block-size threshold behavior established in Section 8.6: with 14,952 payload bits per block, the BLER remains 1.0 through 15 dB, reaches 0.900 at 20 dB, and falls to 0.825 at 25 dB, at which point the uncoded throughput of the profile reaches 2.99 Mbit/s at 20 dB and 5.23 Mbit/s at 25 dB. The Python reference reproduces this campaign on an independent random stream, and the regenerated compact cross-verification record shows every comparison passing at all six SNR points under the tolerances of Section 6.

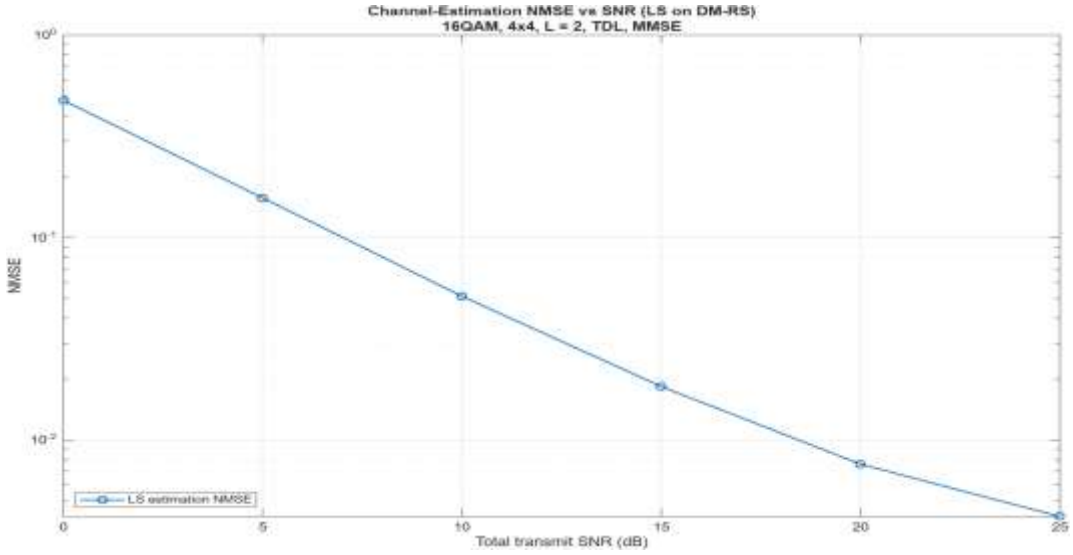

Figure 13. Channel-estimation NMSE against SNR, compact profile, LS estimator of equations (12)-(15).

Figure 13 reports the estimation accuracy directly. The NMSE falls from 4.76×10^{-1} at 0 dB to 4.19×10^{-3} at 25 dB, close to one decade per ten decibels over the grid, which is the noise-floor slope that equation (13) predicts for an unsmoothed LS estimator; the absolute level reflects the two-symbol averaging of equation (14) and the interpolation of equation (15), with constant edge extension, over the TDL-selective response. The agreement between this measured slope and the prediction of equation (13) is the estimation-domain counterpart of the theory anchoring of Section 8.1.

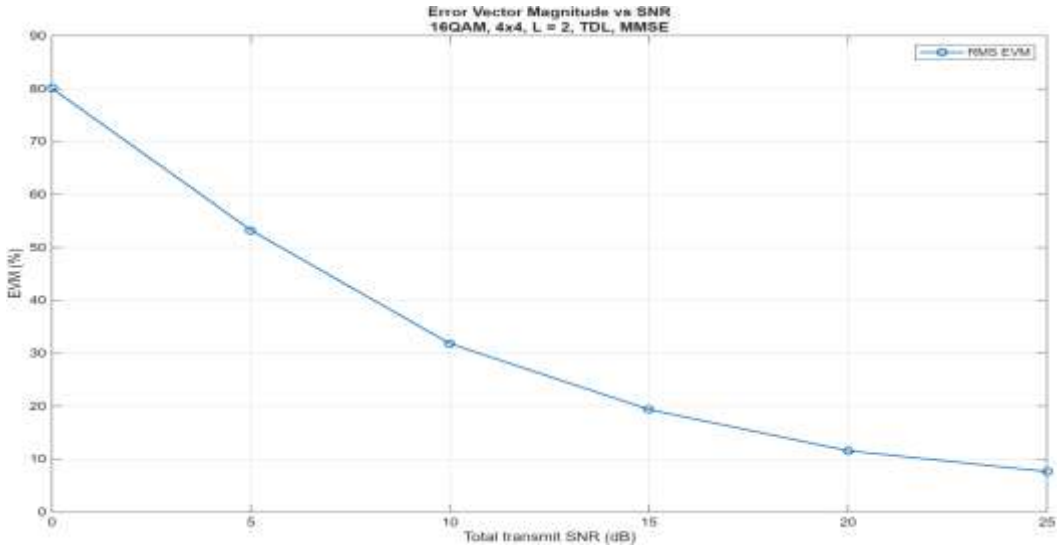

Figure 14. RMS EVM against SNR, compact profile, equation (28).

Figure 14 shows the RMS EVM falling from 80.1% at 0 dB to 7.6% at 25 dB. The EVM aggregates the residual noise, the residual inter-layer interference, and the channel-estimation error of the equalized symbols, so its monotone decrease with the correct slope confirms the composite behavior of equations (24) and (25) on the executed profile.

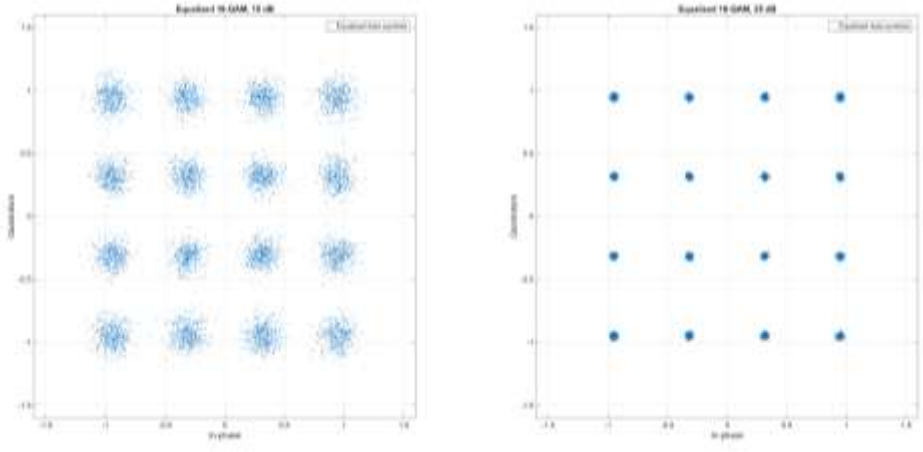

Figure 15. Equalized 16-QAM constellations of the compact profile at low and high SNR.

Figure 15 presents the equalized constellation at the two ends of the sweep. At low SNR the received symbols form an undifferentiated cloud around the origin; at high SNR the sixteen constellation points are fully resolved with visible residual scatter consistent with the measured EVM. The unbiased-MMSE normalization of equation (22) is visible in the

geometry: the constellation is centered on the transmitted symbol positions rather than compressed toward the origin, which is the property that keeps hard-decision boundaries correct for the amplitude-bearing 16-QAM alphabet.

8.6 End-to-End 64×8 Campaign with Adaptive Stopping and Confidence Intervals

Table 4 reports the complete adaptive massive campaign, transport block to CRC decision, with the exact 95% intervals of equation (34) computed from the recorded error counts. The first three columns list the SNR grid, the executed frame count, and the evaluated bits of each point; the following columns give the BER and BLER point estimates with their exact intervals; and the final column records the stopping reason of the adaptive rule. Zero-error entries are stated as upper bounds, and every row uses the executed LS estimator of equations (12)-(15).

Table 4. End-to-end 64×8 adaptive campaign with exact Clopper-Pearson 95% confidence intervals (500 to 20,000 frames per point; 27,624 payload bits per frame).

SNR (dB)	Frames	Bits	BER [95% CI]	BLER [95% CI]	Stop
−10	500	13.8M	3.372×10^{-1} [3.370, 3.375]$\times10^{-1}$	1.000 [0.993, 1.000]	errors
−5	500	13.8M	1.928×10^{-1} [1.926, 1.930]$\times10^{-1}$	1.000 [0.993, 1.000]	errors
0	500	13.8M	7.315×10^{-2} [7.301, 7.329]$\times10^{-2}$	1.000 [0.993, 1.000]	errors
5	500	13.8M	8.372×10^{-3} [8.324, 8.420]$\times10^{-3}$	1.000 [0.993, 1.000]	errors
10	500	13.8M	4.308×10^{-5} [3.969, 4.668]$\times10^{-5}$	0.632 [0.588, 0.674]	errors
15	20,000	552.5M	0 observed; < 6.64×10^{-9}	0 observed; < 1.9×10^{-4}	budget
20	20,000	552.5M	0 observed; < 6.64×10^{-9}	0 observed; < 1.9×10^{-4}	budget

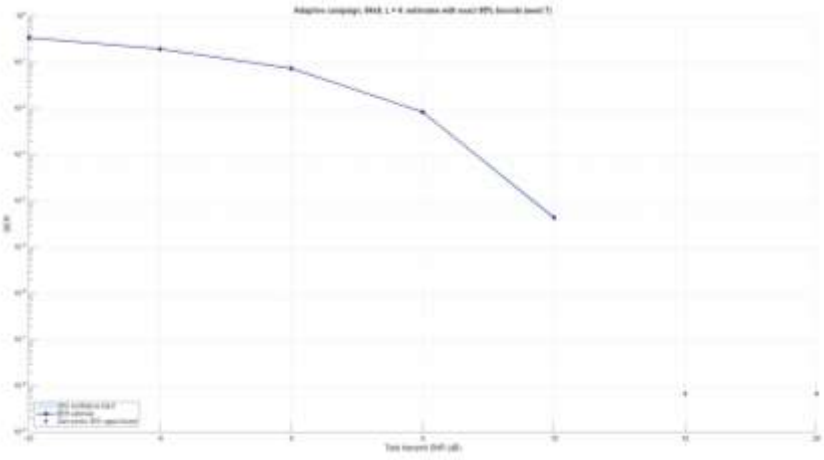

Figure 16. Adaptive-campaign BER against SNR with the exact 95% confidence intervals of Table 4; the two high-SNR points are upper bounds from 552 million evaluated bits each.

Table 4 supports three observations. The measured NMSE of the campaign tracks the pilot noise floor from 1.34 at −10 dB to 1.35×10^{-3} at 20 dB, one decade per ten decibels, exactly the σ_n^2 behavior that equation (13) predicts for an unsmoothed LS estimator; pilot noise rather than interpolation bias therefore dominates the estimation error over the

swept range, with the corresponding EVM falling from 84.4% to 3.4%. The uncoded BLER is a threshold function of the block size: a single residual bit error among 27,624 payload bits fails the CRC, so blocks begin to pass only once the BER approaches the reciprocal of the block size, near 3.6×10^{-5}, which is why the campaign sits mid-cliff at 10 dB where the measured BER is 4.31×10^{-5}. The adaptive rule sets the resolution of every row: the five measured points terminate on their error targets with 500-frame populations, the mid-cliff BLER of 0.632 is known to within [0.588, 0.674], and the two high-SNR points run to the full 20,000-frame budget and bound the BER below 6.64×10^{-9}, three hundred times tighter than a sixty-frame campaign could state.

Cross-implementation agreement completes the verification of the campaign. The independently written Python reference executed the same locked uncoded configuration on an independent random stream, and the regenerated cross-verification record contains thirty-five comparisons, five metrics at seven SNR points, all passing their tolerances: the largest BER deviation across the record is 0.046 decades, at the 10 dB cliff point, with NMSE within 0.004 decades, EVM within 0.5% relative, capacity within 0.5% relative, and BLER within 0.05 absolute at every point. The equal-criteria adaptive campaigns strengthen this further: at every measured point the MATLAB and Python 95% confidence intervals overlap, including the cliff point where the two independent implementations record 4.308×10^{-5} and 4.351×10^{-5}. The Simulink testbench then swept the same seven-point grid as a third execution environment: the verification gates re-ran and passed before every grid point, and all grid points pass against the reference campaign, with worst recorded deviations of 0.009 decades in BER, 0.7% relative in EVM, 0.006 decades in NMSE, 0.45% in capacity, and 0.033 absolute in BLER. Table 5 summarizes the comparison records: each row names one comparison, states the metric classes and tolerances under which it was evaluated, and reports its recorded outcome.

Table 5. Cross-verification and third-environment agreement of the locked 64×8 uncoded configuration.

Comparison	**Metric classes and tolerances**	**Outcome**
MATLAB vs. Python reference, massive profile (independent implementations, independent streams, uncoded chain)	BER 0.3 dec; NMSE 0.15 dec; EVM 10% rel; capacity 2% rel; BLER 0.25 abs	35 of 35 PASS; largest deviation 0.046 dec (BER, 10 dB)
MATLAB vs. Python reference, compact 4×4 TDL profile	Same tolerance set	All comparisons PASS at all six points
MATLAB vs. MATLAB rerun (regression, independent stream)	Same tolerance set	All points within tolerance
Simulink testbench vs. reference campaign (7-point sweep)	Same tolerance set; gates re-run per point	All points PASS; worst BER deviation 0.009 dec

8.7 LDPC-Coded BLER and Throughput

The coded campaigns close the transport chain with the TS 38.212 LDPC code at three target rates, R = 0.30, 0.48, and 0.75, carrying transport blocks of 4,480, 7,168, and 11,272 payload bits on the 14,976 coded bits of the two-layer compact grid. Two campaigns were executed: a coarse campaign on a 5 dB grid with sixty transport blocks per point, and a fine campaign on a 2.5 dB grid with one hundred transport blocks per point.

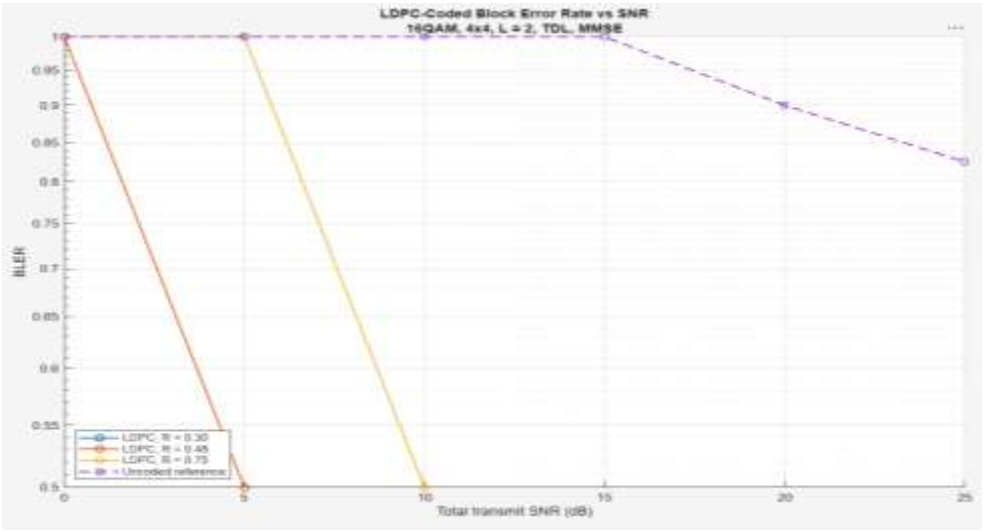

Figure 17. LDPC-coded BLER against SNR, coarse 5 dB grid: 16-QAM, 4×4, L = 2, TDL, MMSE, sixty transport blocks per point, with the uncoded reference. MATLAB campaign.

Figure 17 establishes the coded behavior at coarse resolution: each code rate produces a waterfall that collapses from BLER 1.0 to zero observed errors within one 5 dB step, the cliffs order strictly with the code rate, and the uncoded reference of the same profile passes its first blocks only at 20 dB.

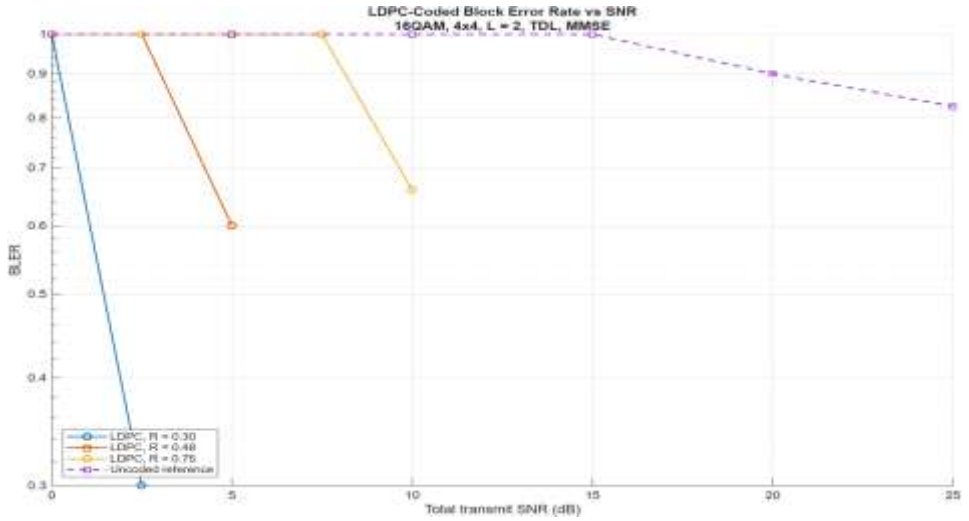

Figure 18. LDPC-coded BLER against SNR, fine 2.5 dB grid: 16-QAM, 4×4, L = 2, TDL, MMSE, one hundred transport blocks per point, with the uncoded reference. MATLAB campaign.

Figure 18 resolves the same waterfalls at 2.5 dB spacing and doubled block count. Each cliff is captured mid-transition, and Table 6 states the executed cliff points together with their exact intervals: for each code rate, the table lists the cliff SNR, the measured BLER at that point, the exact 95% interval of the measurement, and the corresponding mid-cliff BER. Against the uncoded reference, even the highest-rate code closes the link several decibels earlier, and the strongest code approximately twenty decibels earlier.

Table 6. Executed LDPC fine-grid cliff points with exact 95% confidence intervals (100 transport blocks per point). Zero-error points beyond each cliff are bounded by BLER < 0.036.

Code rate	Cliff SNR (dB)	BLER	BLER 95% CI	Mid-cliff BER
R = 0.30	2.5	0.300	[0.212, 0.399]	3.46×10^{-2}
R = 0.48	5.0	0.600	[0.497, 0.697]	3.54×10^{-2}
R = 0.75	10.0	0.660	[0.558, 0.752]	2.00×10^{-2}

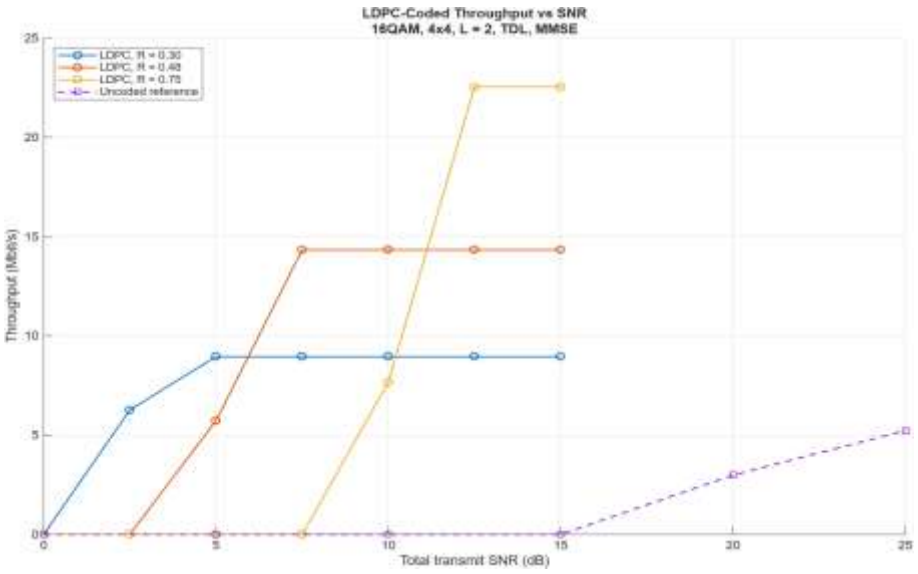

Figure 19. LDPC-coded throughput against SNR, fine grid, 12-RB two-layer allocation.

The throughput of Figure 19 is arithmetically consistent with the block error rate: each curve saturates at its exact plateau of 8.960, 14.336, or 22.544 Mbit/s, the transport-block size divided by the 0.5 ms slot, and every intermediate point equals the plateau multiplied by (1 − BLER) to the printed digit. These plateaus belong to the executed 12-RB allocation and are not extrapolated to wider bandwidths. The crossovers between the curves correspond to link-adaptation operating points: the strongest code delivers the highest throughput at 5 dB, the middle rate dominates the central region, and the highest rate becomes preferable beyond its 10 dB cliff, after which it carries two and a half times the throughput of the strongest code. The coarse and fine grids together also quantify the Monte Carlo resolution of the measurement: at the R = 0.48, 5 dB point the two independent campaigns record BLER 0.500 and 0.600, and the interval [0.497, 0.697] of the larger campaign identifies the discrepancy as statistical rather than implementation-related: the two campaigns sample a steep waterfall region with independent random streams.

8.8 Testbench Execution and RF Instrumentation

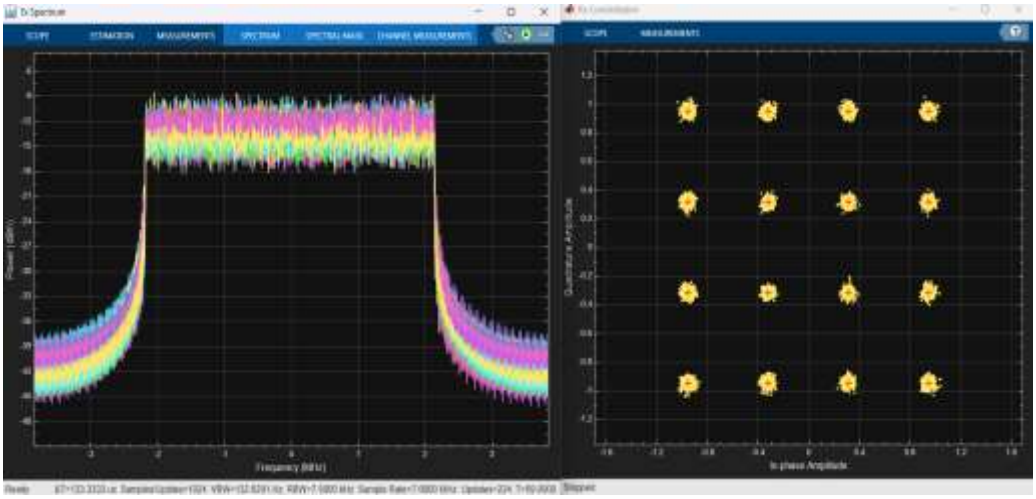

Figure 20. Live testbench instruments during the 64×8 sweep: Spectrum Analyzer on the transmitted CP-OFDM waveform (left) and Constellation Diagram on the equalized layer symbols with the 16-QAM reference points (right).

Figure 20 shows the two testbench instruments during sweep execution. The Spectrum Analyzer displays the transmitted waveform at the 7.68 MHz sample rate with the 64 antenna streams overlaid: the occupied band of approximately 4.3 MHz, the flat in-band level, and the out-of-band skirts correspond to the quantitative Welch estimate of Figure 22, and the instrument state line records the running acquisition (1024 samples per update at 7.68 MHz). The Constellation Diagram displays the equalized layer symbols of the current frame against the sixteen reference points; the capture corresponds to a low-SNR grid point, at which the symbol cloud spreads across the decision regions, consistent with the measured EVM above fifty percent at the low-SNR grid points. The instruments are attached to the passive monitoring branch of Section 5 and consume no random numbers, so their presence leaves every campaign metric unchanged.

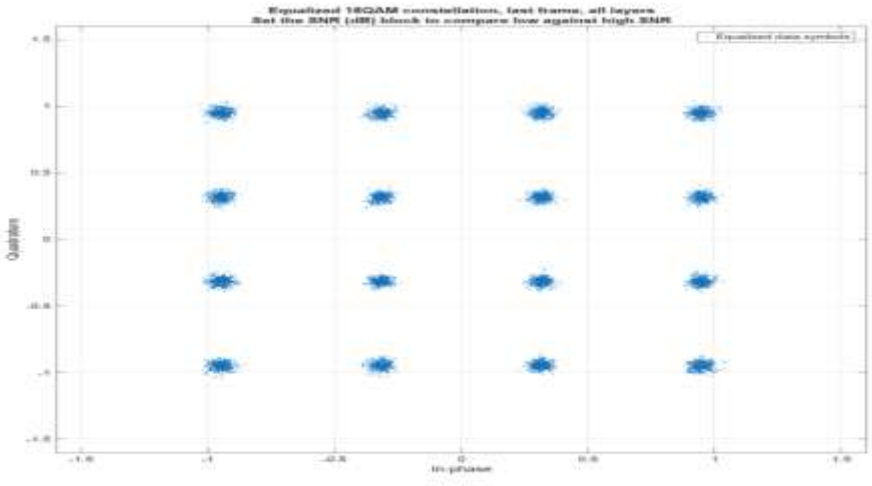

Figure 21. Equalized-symbol constellation logged by the Simulink testbench instrumentation during the 64×8 sweep.

Figure 21 presents the equalized-symbol constellation recorded from the logged testbench data. The view corresponds directly to the per-point EVM column of Table 4:

at the high-SNR grid points the sixteen constellation clusters are fully separated with the residual scatter of the measured 3.4 to 6.1 percent EVM, which is the visual confirmation that the Simulink execution reproduces the receiver operating state of the reference campaign. The numerical confirmation is the sweep record of Table 5: seven of seven grid points pass, with the verification gates re-executed before every point.

8.9 Waveform-Domain Measurements

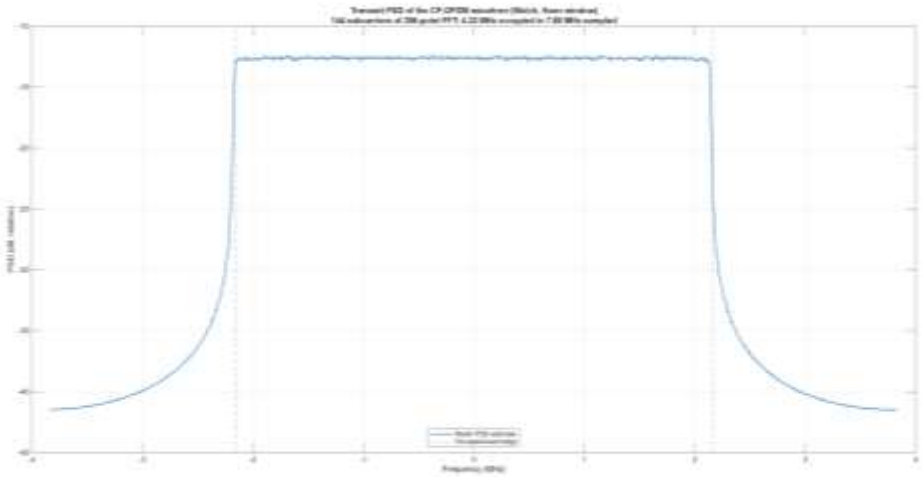

Figure 22. Welch power spectral density of the transmitted CP-OFDM waveform: 144 subcarriers on the 256-point FFT, 4.32 MHz occupied within the 7.68 MHz sampled band.

The testbench logs the transmitted waveform itself, 3,836 samples on 64 antennas over 60 frames at 7.68 MHz, through the passive monitoring branch, which consumes no random numbers and therefore changes no metric result. Figure 22 estimates the PSD with Hann-windowed Welch segments deliberately unaligned to the OFDM symbol boundaries so that the true out-of-band behavior is visible: the 144 active subcarriers occupy 4.32 MHz with in-band ripple within approximately one decibel, and the out-of-band sidelobes fall approximately twenty-eight decibels below the in-band level, the characteristic spectral skirt of rectangular-pulse CP-OFDM without transmit windowing.

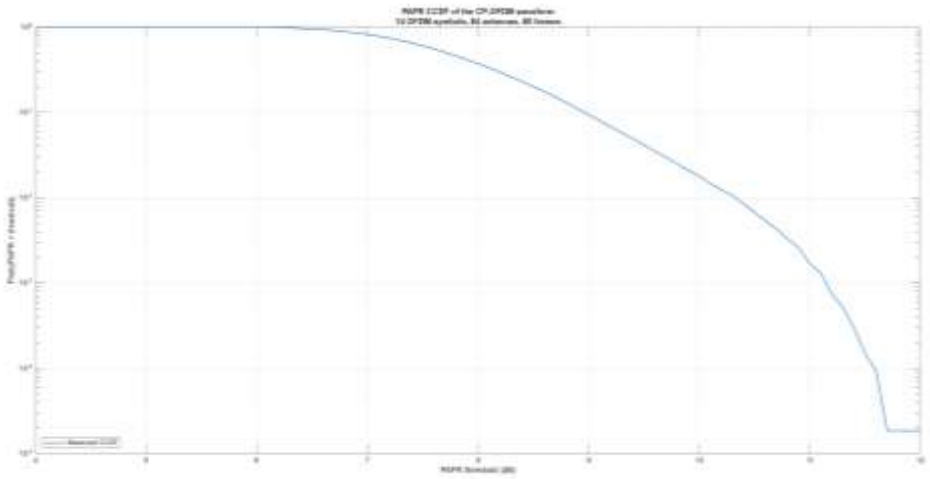

Figure 23. PAPR CCDF of the transmitted CP-OFDM waveform over 53,760 per-antenna OFDM symbol waveforms.

Figure 23 reports the peak-to-average power ratio as a complementary cumulative distribution over the 53,760 per-antenna OFDM symbol waveforms of the run. The PAPR exceeds approximately 9 dB in ten percent and 10.3 dB in one percent of the symbol waveforms, with a worst observed value near 11.7 dB where the distribution reaches the resolution floor set by the reciprocal of the sample count. This transmitter-side quantity is invisible to every frequency-domain metric of the campaigns, and it establishes the measured starting point for RF-aware extension: a power amplifier transmitting this waveform without distortion requires on the order of ten decibels of back-off headroom.

9. Complexity and Reproducibility

This section reports the computational cost of the platform and the contents of the reproducibility package.

9.1 Computational Complexity

The per-frame cost of the chain is dominated by four stages, and their asymptotic costs follow directly from the operations of Sections 3 and 4. The wideband precoder of equation (10) forms the transmit covariance in $O(K N_R N_T^2)$ operations over the K active subcarriers and diagonalizes it once per frame in $O(N_T^3)$, compared with $O(K N_R N_T \min(N_R, N_T))$ for a per-subcarrier SVD; at $N_T = 64$ the single eigendecomposition per frame is a deliberate design decision that keeps the massive profile computationally tractable, and it is also the reason the 64-antenna Simulink testbench executes more slowly per step than the compact 4×4 testbench. The LS estimator of equations (12)-(15) costs $O(K_p N_R L)$ divisions over the K_p pilot subcarriers plus one linear interpolation per antenna-layer pair. The unbiased-MMSE equalizer of equations (21)-(22) builds and inverts one L×L system per subcarrier, $O(K (N_R L^2 + L^3))$, which at $L = 4$ is negligible against the precoder cost. LDPC decoding scales with the iteration count multiplied by the number of edges of the base graph and dominates the coded campaigns. The adaptive campaign records the wall-clock runtime of every result row: at the 64×8 dimensions the MATLAB implementation executes approximately 0.18 s per frame, 86 to 91 s per 500-frame point, and 3,813 s and 3,867 s for the two 20,000-frame points, for a complete seven-point campaign runtime of approximately 2 h 14 min on the executing workstation; the interpreted per-frame eigendecomposition of the 64-antenna covariance dominates this cost, consistent with the asymptotic analysis above. Runtime tables across the antenna scaling, and for the Python and Simulink environments, remain future work under Section 10.

9.2 Reproducibility Package

Every result of Section 8 traces to a stored artifact set under equation (31). The MATLAB package contains the simulation source, the verification-gate scripts, the campaign drivers, the Simulink model with its build scripts, the locked configuration logs, and the per-SNR result files of every executed campaign. The Python package contains the independently written reference modules, the campaign and comparison drivers, the dependency manifests (requirements.txt and pyproject.toml), the license and citation metadata (LICENSE and CITATION.cff), and the result files with the raw bit and block error counts of every Python campaign. The cross-verification records store every comparison of Section 8.6 with its deviation, tolerance, and verdict, and the Simulink sweep record stores the per-point comparison of the testbench execution. All campaigns use fixed, recorded seeds, and every figure of this paper regenerates from its stored result file and the corresponding plotting script. The packages are published under the author's repository at github.com/dipucwc.

10. Limitations and Future Work

The scope limitations are stated explicitly to prevent over-interpretation of the conclusions. The evaluated system is the digital baseband of a single-cell, single-user downlink with ideal timing and frequency synchronization; MAC scheduling, HARQ protocol timing, and higher-layer procedures are excluded, and RF impairments are intentionally excluded from the verified core so that baseband errors cannot be masked by impairment models. The headline massive-array result uses flat i.i.d. Rayleigh fading without spatial correlation or an explicit array geometry, ideal transmitter CSI for the precoder, and the 12-RB reduced allocation; these conditions are appropriate for verification and for isolating the mechanism under study, but they do not support broad practical conclusions about deployed massive MIMO, and the paper claims none. The Python reference covers the uncoded chain only, and the cross-verification is statistical and metric-level rather than sample-by-sample.

The planned extensions are presented in order of priority. The first is experimental breadth: frequency-selective TDL-A and TDL-C profiles at several delay spreads, one line-of-sight case, spatially correlated channels with a defined ULA or UPA geometry and element spacing for the 64-antenna array, low and moderate Doppler, and degraded-CSI conditions including delayed CSI, noisy covariance estimates, and codebook-quantized precoding. The second is comparative rigor: paired Monte Carlo experiments on common stored bits, channels, noise, and pilots for the algorithm comparisons, wideband against subband and per-subcarrier SVD, LS against LMMSE-smoothed estimation, ZF against raw and unbiased MMSE, and pilot-density ablations, with independent-stream runs retained as a separate reproducibility test, together with sample-level MATLAB-Python checkpoint tests on identical stored inputs. The third is external benchmarking: reproduction of a MATLAB 5G Toolbox reference PDSCH configuration and, where the assumptions can be matched exactly, a published reference curve, so that the two internal implementations are additionally anchored to an independent external one. Measured complexity and runtime tables across the antenna scaling complete the program, followed by the research directions of RF-aware simulation, phase noise, IQ imbalance, and power-amplifier nonlinearity starting from the measured PAPR of Figure 23, and learning-based estimation and detection benchmarked against the stored LS, MMSE, and ZF baselines with the statistical qualification of Section 7 applied unchanged.

11. Conclusion

This paper presented a complete PDSCH-oriented 5G NR Massive MIMO-OFDM link-level simulator together with the methodology that qualifies its results for quantitative use: gate-based verification against closed-form references, a tolerance-based statistical cross-verification protocol between the complete MATLAB platform and an independently written uncoded Python reference, a Simulink testbench as a third execution environment for the same verified functions, adaptive Monte Carlo stopping on the principal campaign, and exact confidence intervals attached to every estimate. Every algorithm was documented as a formal derivation, as a numbered procedure with a described flow and a workflow diagram, and as a traceability entry naming its MATLAB, Python, and Simulink implementation, so that the mathematics and the code of the three environments remain aligned throughout. The executed evidence supports the claims within their stated bounds: the uncoded BER lies on closed-form theory across five decades; the LS estimation cost is measured directly as a several-decibel displacement whose NMSE tracks the pilot-noise floor predicted by the estimator error analysis; the eigenbeamformed 64×8 configuration delivers a measured 19.6 dB combined array-dimension and eigenbeamforming gain at BER 10^{-1} over an unprecoded 4×4 reference at equal total transmit power, under the stated flat-Rayleigh, ideal-CSI conditions; the compact frequency-selective campaign confirms the estimator, equalizer, and metric chain end to end with every stored comparison passing; the TS 38.212 LDPC code converts the uncoded CRC threshold into rate-ordered waterfalls with exact throughput plateaus; the adaptive campaign bounds the high-SNR error rates below 6.7×10^{-9} from over half a billion evaluated bits per point; and thirty-five of thirty-five cross-implementation comparisons pass with a largest deviation of 0.046 decades, the Simulink sweep passes at every grid point, and the equal-criteria adaptive campaigns of the two implementations agree within overlapping 95% confidence intervals at every measured point, after the verification protocol itself detected, localized to six subcarriers, and corrected a genuine interpolation edge defect that coarse statistics could not resolve. Beyond the specific results, the paper demonstrates that verification evidence and statistical qualification are essential components of a quantitative link-level study: every figure traces to a stored file, a locked configuration, and a passed gate record, and every error-rate statement is exactly as strong as its executed sample size permits.

12. References

[1] 3GPP TS 38.300, "NR; NR and NG-RAN Overall Description; Stage-2."

[2] 3GPP TS 38.211, "NR; Physical channels and modulation."

[3] 3GPP TS 38.212, "NR; Multiplexing and channel coding."

[4] 3GPP TS 38.214, "NR; Physical layer procedures for data."

[5] 3GPP TR 38.901, "Study on channel model for frequencies from 0.5 to 100 GHz."

[6] 3GPP TS 38.101-1, "NR; User Equipment (UE) radio transmission and reception; Part 1: Range 1 Standalone."

[7] E. Telatar, "Capacity of multi-antenna Gaussian channels," European Transactions on Telecommunications, vol. 10, no. 6, pp. 585-595, 1999.

[8] J. G. Proakis and M. Salehi, Digital Communications, 5th ed. New York, NY, USA: McGraw-Hill, 2008.

[9] A. Goldsmith, Wireless Communications. Cambridge, U.K.: Cambridge University Press, 2005.

[10] D. Tse and P. Viswanath, Fundamentals of Wireless Communication. Cambridge, U.K.: Cambridge University Press, 2005.

[11] E. Dahlman, S. Parkvall, and J. Skold, 5G NR: The Next Generation Wireless Access Technology, 2nd ed. London, U.K.: Academic Press, 2020.

[12] MathWorks, 5G Toolbox: User's Guide. Natick, MA, USA: The MathWorks, Inc.

[13] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357-362, 2020.

[14] S. Pratschner, B. Tahir, L. Marijanovic, M. Mussbah, K. Kirev, R. Nissel, S. Schwarz, and M. Rupp, "Versatile mobile communications simulation: the Vienna 5G Link Level Simulator," EURASIP Journal on Wireless Communications and Networking, vol. 2018, art. 226, 2018.

[15] J. Hoydis, S. Cammerer, F. Ait Aoudia, A. Vem, N. Binder, G. Marcus, and A. Keller, "Sionna: An open-source library for next-generation physical layer research," arXiv:2203.11854, 2022.

[16] N. Nikaein, M. K. Marina, S. Manickam, A. Dawson, R. Knopp, and C. Bonnet, "OpenAirInterface: A flexible platform for 5G research," ACM SIGCOMM Computer Communication Review, vol. 44, no. 5, pp. 33-38, 2014.

[17] T. L. Marzetta, "Noncooperative cellular wireless with unlimited numbers of base station antennas," IEEE Transactions on Wireless Communications, vol. 9, no. 11, pp. 3590-3600, 2010.

[18] E. G. Larsson, O. Edfors, F. Tufvesson, and T. L. Marzetta, "Massive MIMO for next generation wireless systems," IEEE Communications Magazine, vol. 52, no. 2, pp. 186-195, 2014.

[19] F. Rusek, D. Persson, B. K. Lau, E. G. Larsson, T. L. Marzetta, O. Edfors, and F. Tufvesson, "Scaling up MIMO: Opportunities and challenges with very large arrays," IEEE Signal Processing Magazine, vol. 30, no. 1, pp. 40-60, 2013.

[20] E. Björnson, J. Hoydis, and L. Sanguinetti, "Massive MIMO networks: Spectral, energy, and hardware efficiency," Foundations and Trends in Signal Processing, vol. 11, no. 3-4, pp. 154-655, 2017.

[21] J.-J. van de Beek, O. Edfors, M. Sandell, S. K. Wilson, and P. O. Börjesson, "On channel estimation in OFDM systems," in Proc. IEEE Vehicular Technology Conference, 1995, pp. 815-819.

[22] O. Edfors, M. Sandell, J.-J. van de Beek, S. K. Wilson, and P. O. Börjesson, "OFDM channel estimation by singular value decomposition," IEEE Transactions on Communications, vol. 46, no. 7, pp. 931-939, 1998.

[23] S. Coleri, M. Ergen, A. Puri, and A. Bahai, "Channel estimation techniques based on pilot arrangement in OFDM systems," IEEE Transactions on Broadcasting, vol. 48, no. 3, pp. 223-229, 2002.

[24] T. Richardson and S. Kudekar, "Design of low-density parity check codes for 5G new radio," IEEE Communications Magazine, vol. 56, no. 3, pp. 28-34, 2018.

[25] D. J. Love, R. W. Heath, V. K. N. Lau, D. Gesbert, B. D. Rao, and M. Andrews, "An overview of limited feedback in wireless communication systems," IEEE Journal on Selected Areas in Communications, vol. 26, no. 8, pp. 1341-1365, 2008.

[26] P. Vandewalle, J. Kovačević, and M. Vetterli, "Reproducible research in signal processing," IEEE Signal Processing Magazine, vol. 26, no. 3, pp. 37-47, 2009.

[27] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," Biometrika, vol. 26, no. 4, pp. 404-413, 1934.

[28] J. A. Hanley and A. Lippman-Hand, "If nothing goes wrong, is everything all right? Interpreting zero numerators," JAMA, vol. 249, no. 13, pp. 1743-1745, 1983.

[29] M. C. Jeruchim, "Techniques for estimating the bit error rate in the simulation of digital communication systems," IEEE Journal on Selected Areas in Communications, vol. 2, no. 1, pp. 153-170, 1984.